

0. Page de garde

Titre : **IA de détection précoce du décrochage étudiant**

Certification : **Concevoir et implémenter une solution d'intelligence artificielle**

Nom/prénom : **Valeux/marc**

Date : **24/08/2026**

Version : **1.0**

Environnement d'exécution : **python 3.13**

Code source : <https://github.com/marc-proxiel/ProjetDecrochage>

Sommaire

- 0. Page de garde
- 1. Résumé exécutif (problème, objectif, approche, résultats clés, limites)
 - 1.1 Résumé Exécutif : Détection précoce du décrochage étudiant
 - 1.2 Contexte
- 2. Cadrage métier et cas d'usage
 - 2.1 Problématique Métier
 - 2.2 Objectifs de la Solution
 - 2.3 Nature Technique du Cas d'Usage
 - 2.4 Actions Déclenchées
 - 2.5 Les enjeux financiers et optimisation des ressources
 - 2.6 Enjeux sociaux et lutte contre la précarité
 - 2.7 Enjeux institutionnels et d'image
 - 2.8 Résumé des enjeux
 - 2.9 Faux positif ou Faux négatif ?
 - 2.9.1 La priorité au faux négatif (Le risque de l'échec non détecté)
 - 2.9.2 L'enjeu du faux positif (Le risque du marquage et du gaspillage)
 - 2.9.3 Quel arbitrage choisir ?
 - 2.10 Conclusion
- 3. Données : disponibilité, gouvernance et alternatives
 - 3.1 Disponibilité des données
 - 3.2 Gouvernance des données
 - 3.3 Alternatives et enrichissement
- 4. Enjeux éthiques, sociétaux et conformité
 - 4.1 Conformité Réglementaire (RGPD) et Protection de la Vie Privée
 - 4.2 Gestion des Variables Sensibles et des Biais
 - 4.3 Le Dilemme du « Marquage » (Enjeu Sociétal)

- 4.4 Transparence et Explicabilité
- 5. Chargement et compréhension des données
 - 5.1 Le dictionnaire
 - 5.1.1 Identifiants et Administration
 - 5.1.2 Profil Socio-Démographique et Antécédents
 - 5.1.3 Engagement et Activité Numérique
 - 5.1.4 Données Académiques et Perceptions
 - 5.1.5 Variables Leurres
 - 5.1.6 Variables Cibles (Targets)
 - 5.1.7 Enrichissement (Catalogue des formations)
- 6. Analyse exploratoire (EDA) : visualisations et interprétation
 - 6.1 Equilibre de la cible `abandon`
 - 6.2 Distribution de `moyenne_finale` et lien avec l'abandon
 - 6.3 Corrélations avec les variables numériques disponibles à mi-S1
 - 6.4 Taux d'abandon par type de bac
- 7. Préparation des données (nettoyage, manquants, transformations, features)
 - 7.1 Colonnes non pertinentes pour la modélisation -> dataframe silver
 - 7.2 Homogénéisation de la colonne `date_inscription` (dataframe silver)
 - 7.3 Conversion des colonnes numériques mal typées
 - 7.4 Transformation des données
 - 7.5 Homogénéisation de la colonne `bac_type` (dataframe silver)
 - 7.6 Homogénéisation de la colonne `filière` (dataframe silver)
 - 7.7 Homogénéisation de la colonne `mention_bac` (dataframe silver)
 - 7.8 Taux d'abandon global et par filière
 - 7.9 Encodage des variables catégorielles (dataframe silver)
 - 7.10 Bilan des valeurs nulles par colonne (dataframe silver)
 - 7.11 Stratégie d'imputation des valeurs nulles (dataframe silver)
 - 7.12 Dataframe gold
 - 7.12.1 Feature engineering : taux de rendu des devoirs
 - 7.12.2 Vérification : maximum du taux de rendu
 - 7.13 Split train/test et imputation
 - 7.13.1 Split puis imputation (fit sur le train uniquement)
- 8. Choix du modèle et démarche scientifique (baseline, modèles, comparaison)
 - 8.1 Comparaison de plusieurs modèles (classification `abandon`)
 - 8.1.1 Matrices de confusion des 3 modèles
 - 8.1.2 Courbes ROC et Précision-Recall des 3 modèles
 - 8.1.3 Courbe seuil-vs-métriques (choix du seuil de décision)
 - 8.1.4 Courbes de calibration
- 9. Entraînement, validation et ajustement (sélection du modèle final)
 - 9.1 Validation croisée et optimisation des hyperparamètres
 - 9.1.1 Stabilité par validation croisée (modèles par défaut)
 - 9.1.2 Optimisation des hyperparamètres (recherche sur ROC-AUC)
 - 9.1.3 Courbes d'optimisation des hyperparamètres
 - 9.1.4 Re-évaluation des modèles tunes sur le test
 - 9.2 Importance des variables / coefficients (modèles tunes)

- 9.2.1 Diagramme SHAP (régression logistique retenue)
- 9.3 Modèle retenu : régression logistique
- 9.4 Suivi des configurations avec MLflow
 - 9.4.1 Journalisation des configurations par défaut (baseline)
 - 9.4.2 Journalisation des configurations tunees, et enregistrement du modèle retenu
- 9.5 Prédiction de moyenne finale
 - 9.5.1 Préparation et comparaison de modèles
 - 9.5.2 Validation croisée et optimisation des hyperparamètres
 - 9.5.3 Re-évaluation des modèles tunes sur le test
 - 9.5.4 Modèle retenu pour la prédiction de moyenne finale
 - 9.5.5 Suivi des configurations avec MLflow (régression moyenne_finale)
- 9.6 Peut-on dériver l'abandon à partir de la note prédite ?
- 9.7 Fonction combinée : proba d'abandon + note prédite
- 9.8 Entraînement final des modèles retenus (abandon + moyenne_finale)
 - 9.8.1 Classifieur (abandon)
 - 9.8.2 Régresseur (moyenne_finale)
 - 9.8.3 Métadonnées (les deux modèles)
- 10. Implémentation et mise en exploitation (déploiement, exemple d'usage)
 - 10.1 API : servir les deux modèles ensemble
 - 10.2 Contrat d'entrée/sortie (schémas.py)
 - 10.3 Préparation d'une ligne, côté serveur (main.py)
 - 10.4 Démonstration (appel réel via TestClient, sans lancer de serveur séparé)
 - 10.5 Déployer et utiliser l'API via Docker (docker-compose)
 - 10.5.1 Démarrer la stack
 - 10.5.2 Utiliser l'API (conteneur)
 - 10.5.3 Arrêter la stack
 - 10.6 Alertes Grafana : exploitation et dérive des modèles
 - 10.6.1 Alertes d'exploitation (l'API répond-elle correctement ?)
 - 10.6.2 Alertes de dérive (les prédictions restent-elles dans la plage attendue ?)
 - 10.6.3 Vérification (état réel des règles, via l'API Grafana)
- 11. Architecture cible et contraintes
 - 11.1 Architecture cible pour la Direction de la réussite étudiante
 - 11.1.1 Sources de données
 - 11.1.2 Pipeline batch (ré-entraînement périodique)
 - 11.1.3 Service d'inférence
 - 11.1.4 Orchestration applicative (le chaînon manquant -> implémenté ci-dessous)
 - 11.1.5 Interface pour la Direction de la réussite étudiante
 - 11.1.6 Monitoring & gouvernance
 - 11.1.7 Conformité (cadrée éthiquement au chapitre 4, pas encore implémentée techniquement)
 - 11.1.8 Implémentation : scoring batch -> PostgreSQL
 - 11.2 Contraintes de l'architecture cible
 - 11.2.1 Réglementaires (RGPD, déjà cadrées chapitre 4)
 - 11.2.2 Éthiques (dilemme du marquage, déjà discuté chapitre 2)
 - 11.2.3 Capacité opérationnelle (contrainte concrète, pas juste technique)

- 11.2.4 Calendrier académique (fraicheur des données)
- 11.2.5 Intégration SI (hors de notre contrôle)
- 11.2.6 Portage dans la durée
- 11.2.7 Confiance et adoption
- 11.3 Schémas d'architecture : pilote et exploitation cible
 - 11.3.1 Schéma 1 : le pilote (ce qui a été construit et testé dans ce notebook)
 - 11.3.2 Schéma 2 : exploitation cible (avec la Direction de la réussite étudiante)
- 12. Mesure de performance et impacts (métriques techniques + métier)
 - 12.1 Explicabilité individuelle : pourquoi CET étudiant précis est signalé
 - 12.1.1 Comment lire ce graphique (explication côté métier)
- 13. Amélioration continue (ré-entraînement, suivi, versioning)
 - 13.1 Re-entraînement
 - 13.2 Suivi (monitoring)
 - 13.3 Versioning
- 14. Conclusion et recommandations
 - 14.1 Conclusion
 - 14.2 Recommandations
 - 14.3 Coûts et retour sur investissement (ROI) : seuil 0.5 vs seuil 0.33
- 15. Annexe
 - 15.1 Coût d'un redoublement
 - 15.1.1 Coût direct par étudiant pour l'université
 - 15.1.2 Financement de ces coûts
 - 15.1.3 Impact du redoublement sur le budget de l'université
 - 15.1.4 Stratégies des universités pour limiter les coûts
 - 15.1.5 Comparaison : Coût pour l'université vs. coût pour l'étudiant
 - 15.1.6 Enjeux actuels (2026)
 - 15.2 Model Card

1. Résumé exécutif (problème, objectif, approche, résultats clés, limites)

1.1 Résumé Exécutif : Détection précoce du décrochage étudiant

- **Problème** L'université fait face à un **taux d'abandon élevé en première année de licence (L1)**, estimé à environ **28 %** dans le jeu de données actuel. Certaines filières, comme la Biologie ou l'Informatique, affichent des taux de réussite historiques ne dépassant pas 60 %. L'enjeu est d'intervenir avant que le décrochage ne soit définitif, car les difficultés non détectées représentent un coût important pour l'institution et l'étudiant.
- **Objectif** L'objectif principal est de concevoir une solution d'IA capable de **prédire le risque de décrochage** (classification binaire) dès le milieu du premier semestre. En complément, le modèle doit **estimer la moyenne finale** (régression) pour permettre à la Direction de la réussite

étudiante de prioriser et de calibrer les dispositifs d'accompagnement, tels que le tutorat ou l'aide sociale.

- **Approche** Le projet suit une démarche de bout en bout (compétences C1 à C9) intégrée dans un **notebook Jupyter unique**. L'approche repose sur :
 - Le **traitement de données brutes** complexes (doublons, valeurs manquantes, formats hétérogènes) et l'exclusion de **variables "leurres"** sans pouvoir prédictif.
 - Une rigueur méthodologique stricte pour éviter les **fuites de données** (*data leakage*), en excluant notamment la moyenne finale et les résultats consolidés de fin de semestre lors de la phase de prédiction à mi-parcours.
 - La comparaison de plusieurs familles de modèles (ex: régression logistique, arbres, boosting) pour sélectionner la meilleure performance.
- **Résultats clés attendus**
 - **Performance technique** : Un modèle validé par sa **courbe ROC et son AUC**, ainsi qu'une analyse de la matrice de confusion.
 - **Indicateurs métiers** : Une justification du seuil de classification basée sur le **coût d'un faux négatif** (étudiant à risque non détecté).
 - **Explicabilité** : Fournir aux équipes pédagogiques une lecture claire des facteurs de risque (ex: importance des variables).
 - **Industrialisation** : Une architecture cible incluant un plan de monitoring (MLOps) pour détecter la dérive des données dans le temps.
- **Limites et Risques**
 - **Éthique et Biais** : L'utilisation de **variables sensibles** (sexe, statut de boursier) présente un risque de biais discriminatoires qu'il convient d'analyser.
 - **Dilemme du marquage** : Le risque de stigmatisation des étudiants identifiés comme "à risque" doit être géré par des garde-fous éthiques.
 - **Qualité des données** : La présence de près de 12 % de données manquantes sur certaines variables clés et le caractère partiel des données à mi-S1 peuvent limiter la précision du modèle.

1.2 Contexte

Une université pluridisciplinaire constate un taux d'abandon élevé en première année de licence (L1). La direction de la réussite étudiante souhaite déployer une solution d'intelligence artificielle de détection précoce du décrochage afin d'identifier, **dès le milieu du premier semestre**, les étudiants à risque et de déclencher des dispositifs d'accompagnement (tutorat, soutien méthodologique, aide sociale). Les objectifs métier de la solution sont les suivants :

- prédire, à mi-parcours, le risque de décrochage (interruption de la formation) d'un étudiant à partir de ses données académiques, d'engagement et de contexte ;
- estimer la moyenne finale attendue afin de prioriser et calibrer l'accompagnement ;
- fournir aux équipes pédagogiques des indicateurs explicables, tout en garantissant un usage éthique et conforme des données étudiantes.

2. Cadrage métier et cas d'usage

Le **cadrage métier** et le **cas d'usage** de ce projet concernent la mise en place d'une solution d'intelligence artificielle pour la **détection précoce du décrochage étudiant** au sein d'une université pluridisciplinaire.

Voici les composantes détaillées de ce cadrage :

2.1 Problématique Métier

L'université fait face à un **taux d'abandon élevé en première année de licence (L1)**. Les données indiquent un taux d'abandon moyen d'environ **28 %**. L'enjeu est de pouvoir intervenir de manière proactive dès le **milieu du premier semestre (mi-S1)**, moment où les dispositifs d'accompagnement peuvent encore être efficaces pour redresser la trajectoire de l'étudiant.

2.2 Objectifs de la Solution

La direction de la réussite étudiante a défini trois objectifs majeurs pour cet outil :

- **Prédire le risque de décrochage** : Identifier, à mi-parcours, les étudiants susceptibles d'interrompre leur formation (classification binaire).
- **Estimer la performance académique** : Prédire la moyenne finale attendue afin de **prioriser et calibrer l'accompagnement** nécessaire (régression).
- **Aider à la décision pédagogique** : Fournir des indicateurs explicables aux enseignants pour qu'ils comprennent les facteurs de risque, tout en respectant l'éthique et le RGPD.

2.3 Nature Technique du Cas d'Usage

Il s'agit d'un problème d'**apprentissage supervisé mixte** :

- **Cible principale (Classification)** : Variable `abandon` (0 ou 1).
- **Cible secondaire (Régression)** : Variable `moyenne_finale` (note sur 20).
- **Périmètre temporel** : Le scoring doit être réalisé exclusivement avec des données disponibles à la mi-S1 pour éviter toute **fuite temporelle** (ne pas utiliser les résultats consolidés de fin de semestre comme `moyenne_partiels_s1`).

2.4 Actions Déclenchées

La détection d'un risque élevé doit permettre de déclencher des leviers spécifiques :

- **Tutorat** par des pairs.
- **Soutien méthodologique** pour l'organisation du travail.
- **Aide sociale** pour les étudiants en difficulté financière (le statut de boursier ou les heures de travail rémunéré étant des facteurs d'influence).

2.5 Les enjeux financiers et optimisation des ressources

L'échec et le décrochage représentent un coût significatif pour l'institution :

Coût de l'échec non détecté : Dans l'évaluation de la solution, l'université identifie explicitement le « coût d'un faux négatif ». Un étudiant en situation de décrochage qui n'est pas soutenu à temps représente un investissement pédagogique et administratif perdu pour l'année en cours. Optimisation des dispositifs d'aide : La direction de la réussite étudiante doit « calibrer l'accompagnement ». Identifier précisément les besoins permet de mieux répartir les budgets alloués au tutorat, au soutien méthodologique et à l'aide sociale, évitant ainsi le gaspillage de ressources sur des profils ne présentant pas de risques réels. Gestion des capacités d'accueil : Le catalogue des formations montre des capacités d'accueil limitées (ex: 980 places en Informatique, 830 en Droit). Un taux d'abandon élevé (actuellement autour de 28 %) signifie qu'une part importante de ces places financées ne débouche sur aucun diplôme, réduisant l'efficacité du service public

2.6 Enjeux sociaux et lutte contre la précarité

- Étudiants salariés et boursiers : Une partie des étudiants travaille jusqu'à 30 heures par semaine en parallèle de leurs études. L'échec académique pour ces profils, souvent boursiers, peut aggraver leur précarité financière et compromettre leur avenir professionnel.
- Mobilité sociale : En intervenant dès le milieu du premier semestre, l'université cherche à éviter que des facteurs socio-économiques (comme la distance domicile-campus ou la charge de travail rémunéré) ne deviennent des barrières insurmontables à la réussite

2.7 Enjeux institutionnels et d'image

- Missions de la « Direction de la réussite étudiante » : Cette entité a pour mission directe de redresser les taux de réussite historiques qui sont parfois faibles (60 % en Biologie ou Informatique). Améliorer ces indicateurs est crucial pour l'attractivité de l'université.
- Éthique et bien-être : L'université se préoccupe de l'impact psychologique du décrochage. Elle doit éviter qu'une détection de risque ne stigmatise l'étudiant, tout en renforçant son sentiment d'appartenance et sa satisfaction vis-à-vis de la formation

Un faux positif correspond à un étudiant qui est prédit à tort comme étant en risque de décrochage. Le coût pour l'institution se décline selon trois axes principaux :

- Le gaspillage de ressources : L'université mobilise inutilement des « dispositifs d'accompagnement » tels que le tutorat, le soutien méthodologique ou l'aide sociale pour des étudiants qui n'en ont pas réellement besoin. Cela représente une perte d'efficacité budgétaire et humaine, car ces ressources sont limitées et pourraient être mieux allouées à des profils réellement fragiles.
- Le « dilemme du marquage » : C'est un enjeu éthique. Identifier à tort un étudiant comme étant « à risque » peut avoir des conséquences psychologiques négatives, créer un sentiment de stigmatisation ou modifier la perception que l'étudiant a de ses propres capacités, ce qui pourrait paradoxalement nuire à son parcours.

- La saturation des capacités d'accueil : les filières ont des capacités d'accueil limitées pour ne pas encombrer les dispositifs de soutien au détriment de ceux qui en dépendent pour réussir.

2.8 Résumé des enjeux

Éviter l'échec est un levier pour rentabiliser l'investissement public, assurer la mission de promotion sociale de l'université et garantir la pérennité de ses filières les plus sous tension. **Éviter ces faux négatifs** est au cœur de la mission de la « Direction de la réussite étudiante », car chaque abandon représente une perte par rapport aux capacités d'accueil limitées des filières et aux taux de réussite historiques à redresser.

2.9 Faux positif ou Faux négatif ?

D'après les sources, il n'y a pas de réponse unique, car le choix dépend de l'arbitrage politique et budgétaire de l'université. Cependant, l'énoncé de la certification met un accent particulier sur la gestion des **faux négatifs**, tout en soulignant les risques éthiques des **faux positifs**.

2.9.1 La priorité au faux négatif (Le risque de l'échec non détecté)

Un faux négatif est un étudiant qui va réellement décrocher, mais que l'IA a classé comme "non à risque".

- **L'impact majeur** : C'est l'étudiant qui "passe entre les mailles du filet". L'université manque alors l'opportunité de déclencher les dispositifs de soutien (tutorat, soutien méthodologique, aide sociale) au moment crucial de la mi-parcours.
- **Choix du seuil** : Il faudra par la suite « **justifier le choix du seuil au regard du coût métier d'un faux négatif** ». Cela indique que, pour la Direction de la réussite étudiante, le coût de l'inaction face à un décrochage réel est un indicateur de performance clé.

2.9.2 L'enjeu du faux positif (Le risque du marquage et du gaspillage)

Un faux positif est un étudiant qui aurait réussi seul, mais qui est prédit comme étant en risque de décrochage.

- **Le dilemme éthique** : Les sources évoquent le « **dilemme du marquage** ». Identifier à tort un étudiant comme étant "en difficulté" peut être stigmatisant et influencer négativement sa perception de lui-même ou celle des enseignants.
- **L'optimisation des ressources** : L'université dispose de capacités d'accueil et de ressources limitées (ex: budget pour le tutorat). Trop de faux positifs entraîneraient un gaspillage de ressources pédagogiques et financières sur des étudiants qui n'en ont pas besoin, au risque de saturer les dispositifs pour ceux qui sont réellement fragiles.

2.9.3 Quel arbitrage choisir ?

On utilisera la **matrice de confusion** et la **courbe ROC/AUC** pour fixer un seuil.

- Si l'université considère que **chaque abandon est un échec institutionnel grave**, elle préférera réduire les **faux négatifs** (favoriser le *rappel*), quitte à accepter quelques faux positifs.
- Si les ressources d'accompagnement sont très limitées ou si la crainte de la stigmatisation est forte, elle cherchera à limiter les **faux positifs** (favoriser la *précision*).

Ne connaissant pas les contraintes métiers on prendra la première hypothèse de **minimiser les faux négatifs** qui restent pour l'université et pour l'étudiant l'objectif le plus important.

2.10 Conclusion

Ne connaissant pas les contraintes métiers et financières, on prendra la première hypothèse de **minimiser les faux négatifs** qui restent pour l'université et pour l'étudiant l'objectif le plus important.

3. Données : disponibilité, gouvernance et alternatives

3.1 Disponibilité des données

L'université met à disposition trois ressources principales pour ce cas d'usage :

- **Le jeu de données principal (`decrochage_etudiants_complet_V5.csv`)** : Il contient environ **5 240 lignes** et **33 colonnes**. Il concerne les étudiants de L1 de l'année 2024-2025, observés à mi-parcours du premier semestre.
- **Un échantillon de test (`decrochage_etudiants_echantillon_V5.csv`)** : Une version réduite de 50 lignes pour une première lecture et l'exploration initiale.
- **Le catalogue des formations (`catalogue_formations_V5.csv`)** : Une table de référence décrivant les filières (faculté, capacité d'accueil, taux de réussite historique, etc.) utilisable pour enrichir le jeu principal.

Note sur la qualité : Bien que disponibles, ces données sont "volontairement brutes" et présentent des manquants (jusqu'à 12 % sur certaines variables) et des doublons (40 lignes exactes).

3.2 Gouvernance des données

La gouvernance s'articule autour de la conformité réglementaire et de l'éthique (Compétence C2) :

- **Respect du RGPD** : L'usage des données étudiantes doit être "éthique et conforme". Cela implique l'exclusion des identifiants directs (`student_id` , `id_dossier`) des modèles pour garantir l'anonymat.
- **Gestion des variables sensibles** : Les variables comme le `sexe` ou le statut `boursier` sont identifiées comme sensibles. La gouvernance impose d'analyser les biais possibles (sur ou sous-représentation) pour éviter toute discrimination algorithmique.

- **Cycle de vie et stockage** : Le projet doit documenter le choix de stockage et le cycle de vie des données dans le notebook.

3.3 Alternatives et enrichissement

Face aux imperfections du jeu de données initial, plusieurs alternatives sont prévues :

- **Enrichissement autorisé** : Vous avez explicitement l'autorisation d'**enrichir, filtrer ou ré-échantillonner** les données pour améliorer le modèle.
- **Source secondaire interne** : L'intégration du catalogue des formations est la première alternative recommandée pour pallier le manque de contexte par filière.
- **Stratégies de traitement** : En cas de données manquantes, il est demandé d'envisager des alternatives méthodologiques (stratégies d'imputation documentées dans le journal de bord).
- **Évolution du jeu de données** : L'énoncé précise que le jeu de données est "léger" et que vous êtes invité à le faire évoluer, ce qui peut inclure l'utilisation de données publiques (Open Data) comme évoqué précédemment dans notre conversation.

Toute décision concernant ces aspects (choix d'une alternative, exclusion d'une variable ou ajout d'une source externe) doit être impérativement consignée dans le **journal de bord** intégré au notebook.

4. Enjeux éthiques, sociétaux et conformité

Les enjeux éthiques, sociétaux et de conformité sont centraux. Ils s'articulent autour de quatre axes majeurs :

4.1 Conformité Réglementaire (RGPD) et Protection de la Vie Privée

L'université doit garantir un usage des données scolaires à la fois **éthique et conforme** à la réglementation sur la protection des données (RGPD).

- **Anonymisation et Exclusion** : Pour respecter la confidentialité, les identifiants directs tels que `student_id` (identifiant anonyme de l'étudiant) et `id_dossier` (numéro administratif) doivent être impérativement **exclus des modèles de prédiction**.
- **Gouvernance** : Le projet impose de documenter le choix de stockage et le cycle de vie des données pour assurer une traçabilité conforme.

4.2 Gestion des Variables Sensibles et des Biais

L'analyse doit porter une attention particulière aux variables dites « sensibles » afin d'éviter toute discrimination.

- **Identification des Risques** : Les variables **sexe** (avec des encodages multiples comme f, h, Femme...) et **boursier** (statut boursier) sont explicitement identifiées comme présentant des **risques de biais**.
- **Analyse de Représentativité** : Il est nécessaire d'étudier les phénomènes de **sur-représentation** ou de **sous-représentation** au sein du jeu de données pour s'assurer que le modèle ne pénalise pas injustement certains groupes d'étudiants (ex: biais de genre ou lié à l'origine sociale).

4.3 Le Dilemme du « Marquage » (Enjeu Sociétal)

L'un des défis sociétaux les plus complexes est celui de l'étiquetage des étudiants.

- **Stigmatisation** : Identifier préventivement un étudiant comme étant « à risque » de décrochage peut induire un effet de stigmatisation, impactant sa propre perception de ses capacités ou celle de ses enseignants.
- **Impact Psychologique** : L'université doit arbitrer entre le bénéfice d'une aide précoce (tutorat, aide sociale) et le risque psychologique lié à ce marquage.

4.4 Transparence et Explicabilité

Pour que la solution soit acceptée et utile, elle ne doit pas fonctionner comme une « boîte noire ».

- **Aide à la Décision** : Le modèle doit fournir aux équipes pédagogiques des **indicateurs explicables** (ex: via l'importance des variables ou SHAP) pour justifier pourquoi un étudiant est considéré à risque.
- **Information des Parties Prenantes** : Des garde-fous doivent être mis en place pour informer les étudiants et les enseignants sur le fonctionnement et les finalités de l'outil d'IA.

5. Chargement et compréhension des données

5.1 Le dictionnaire

Voici le dictionnaire de données complet du projet, synthétisé à partir du jeu de données principal et du catalogue des formations. Le fichier principal contient **33 colonnes** relatives à l'année universitaire 2024-2025.

On fait l'hypothèse que les données sont recueillies en milieu de 1er semestre pour éviter une fuite de données pour les colonnes d'"Engagement et Activité Numérique" et "Données Académiques et Perceptions" sauf exceptions mentionnées ci-après.

5.1.1 Identifiants et Administration

Ces variables servent à la gestion des dossiers mais doivent être exclues de la modélisation pour éviter le sur-apprentissage ou par respect de l'anonymat (RGPD).

Variable	Type	Description / Remarques Qualité
student_id	Texte	Identifiant anonyme (ex: ETU-00001). À exclure du modèle (RGPD).
id_dossier	Entier	Numéro administratif à 6 chiffres. À exclure du modèle (RGPD).
année_universitaire	Texte	Constante (2024-2025). À exclure du modèle
niveau	Texte	Constante (L1). À exclure du modèle
date_inscription	Date	Formats mélangés (AAAA-MM-JJ, JJ/MM/AAAA, etc.). Parsing requis.

5.1.2 Profil Socio-Démographique et Antécédents

Variables décrivant le contexte personnel et le parcours scolaire de l'étudiant.

Variable	Type	Description / Remarques Qualité
sexe	Texte	F / M / Autre. Variable sensible (risque de biais).
age	Entier	Âge (17–27 ans).
boursier	Binaire	Statut boursier (0/1 ou Oui/Non). Variable sensible.
bac_type	Texte	Type de bac (général, techno, pro). Prédicteur important.
mention_bac	Texte	Mention (Passable à TB). ~4 % de données manquantes.
établissement_origine	Texte	Type de lycée d'origine. ~5 % de données manquantes.
distance_domicile_km	Décimal	Distance campus (0–140 km). Unités et virgules à nettoyer.
heures_travail_rémunéré_sem	Entier	Heures travaillées / semaine (0–30). ~12 % de manquants.

5.1.3 Engagement et Activité Numérique

Indicateurs de l'implication de l'étudiant, notamment sur la plateforme LMS.

Variable	Type	Description / Remarques Qualité
taux_présence_pct	Décimal	Présence en cours. Formats variés ("85%", "85,0") à convertir.
connexions_lms_30j	Entier	Nombre de connexions sur les 30 derniers jours.
heures_lms_total	Décimal	Temps cumulé sur le LMS (0–180 h).
ressources_consultées	Entier	Nombre de ressources pédagogiques consultées.
messages_forum	Entier	Nombre de messages postés.
nb_devoirs_total	Entier	Devoirs attendus (8–13).
nb_devoirs_rendus	Entier	Devoirs rendus. Utile pour créer un "taux de rendu" . On fait l'hypothèse que ces données sont recueillies en milieu de 1er semestre pour éviter une fuite de données. On vérifiera cela lors de la génération du gold dataset.
retards_rendus	Entier	Nombre de rendus hors délais.

5.1.4 Données Académiques et Perceptions

Ces données reflètent le niveau et le ressenti de l'étudiant à mi-parcours.

Variable	Type	Description / Remarques Qualité
filière	Texte	Mention de L1. Casse et espaces à normaliser.
moyenne_partiels_s1	Décimal	Fuite temporelle : Information consolidée en fin de S1. Hors périmètre pour le scoring à mi-S1.
nb_ue_total / validees_s1	Entier	Fuite temporelle : Nombre d'Unités d'Enseignement et validation (fin de S1).
motivation / satisfaction / sentiment_appartenance	Entier (1-5)	Auto-évaluations. Entre 5 et 8 % de données manquantes. On fait l'hypothèse que ces données sont recueillies en milieu de 1er semestre pour éviter une fuite de données.
commentaire_tuteur	Texte	Fuite temporelle : Commentaire libre. ~49 % de vide. Il est fortement probable que les commentaires soient rédigés en fin de semestre (ex de commentaire : "Bon semestre"). Ils seront exclus du modèle.

5.1.5 Variables Leurres

Ces trois variables n'ont **aucun pouvoir prédictif** et servent à tester votre rigueur lors de la sélection des caractéristiques :

- **groupe_td** (A–H)
- **couleur_carte_étudiante** (bleu, vert, etc.)
- **jour_inscription** (lundi, mardi, etc.)

5.1.6 Variables Cibles (Targets)

- **abandon (0/1)** : Cible principale de classification. 1 = décrochage.
- **moyenne_finale (/20)** : Cible secondaire de régression. **Attention** : Ne doit jamais servir de variable explicative pour prédire l'abandon (**fuite de données**).

5.1.7 Enrichissement (Catalogue des formations)

On joindra ces données via la colonne **filiere** pour enrichir le modèle :

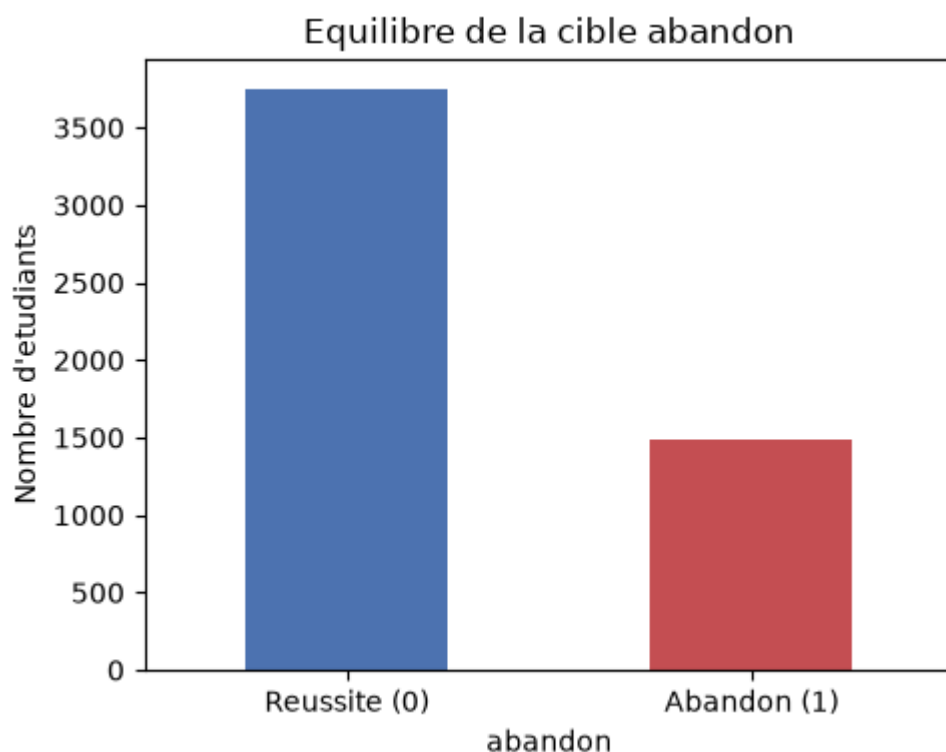
- **faculté** : Composante de rattachement.
- **capacité_accueil** : Nombre de places disponibles.
- **volume_horaire_s1** : Charge de travail prévue.
- **taux_réussite_historique_pct** : Performance passée de la filière.

(5240, 39)

6. Analyse exploratoire (EDA) : visualisations et interprétation

Exploration visuelle a partir de `df_bronze` (avant le nettoyage approfondi fait en 8.7) : équilibre de la cible, lien entre les variables d'engagement et l'abandon, corrélations, et écarts par `bac_type`. Les colonnes encore mal types (`distance_domicile_km` , `taux_presence_pct` , `filiere` , `bac_type`) sont nettoyées dans une copie locale `df_eda` , sans modifier `df_bronze` ni anticiper le pipeline de nettoyage officiel.

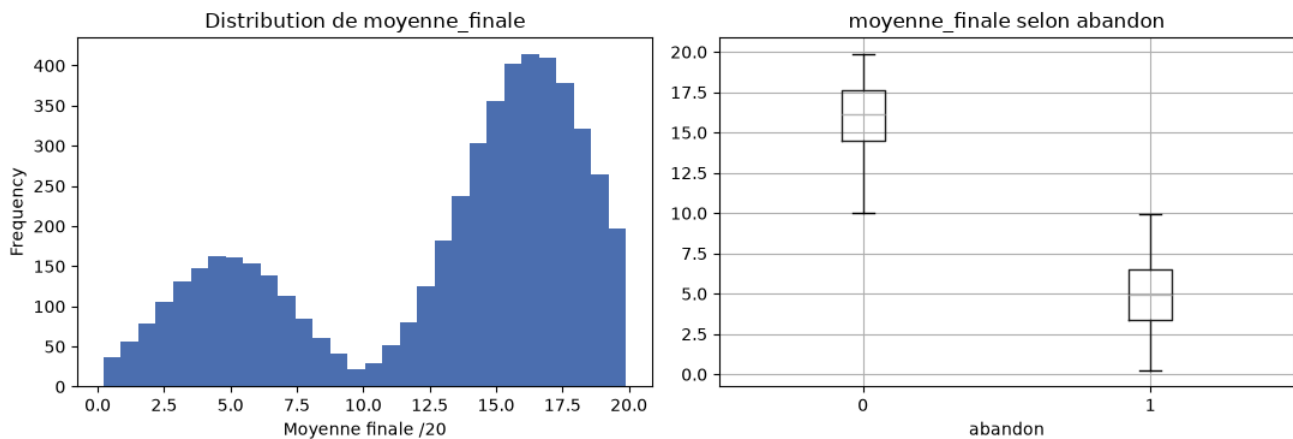
6.1 Equilibre de la cible `abandon`



```
abandon
0    71.58
1    28.42
Name: proportion, dtype: float64
```

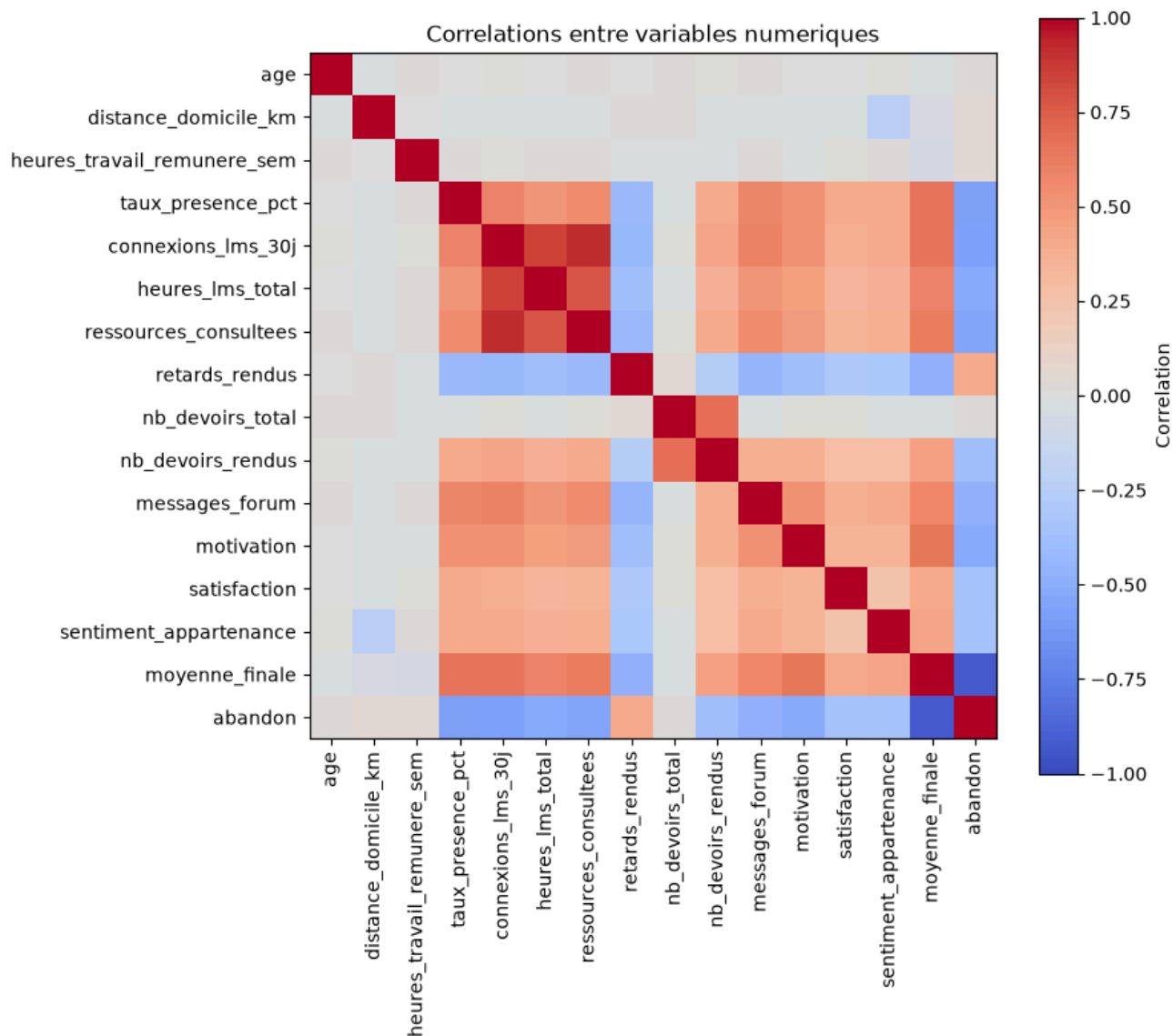
Interprétation. La cible est déséquilibrée mais pas extrême : 71.6 % de réussite contre 28.4 % d'abandon. C'est suffisamment marqué pour qu'un modèle qui prédirait toujours "pas d'abandon" atteigne 71.6 % d'accuracy sans aucune valeur predictive : il faudra évaluer avec le rappel, le F1 ou l'AUC plutôt qu'avec la seule accuracy, cohérent avec la priorité donnée plus haut à la réduction des faux négatifs.

6.2 Distribution de `moyenne_finale` et lien avec l'abandon



Interprétation. Sans surprise, les étudiants en situation d'abandon (`abandon = 1`) ont une `moyenne_finale` nettement plus basse (corrélation de -0.92 avec `abandon`). C'est attendu puisque les deux variables décrivent la même réalité académique en fin de semestre : à ce titre, `moyenne_finale` reste utile pour l'EDA et pour la cible de régression secondaire, mais elle ne doit jamais servir de variable explicative pour prédire `abandon` (fuite de données déjà signalée au chapitre 5).

6.3 Corrélations avec les variables numériques disponibles à mi-S1



```

moyenne_finale      -0.918157
taux_presence_pct    -0.578791
connexions_lms_30j   -0.573985
ressources_consultees -0.536337
motivation           -0.521174
heures_lms_total     -0.503792
messages_forum       -0.474205
nb_devoirs_rendus    -0.387101
satisfaction         -0.353333
sentiment_appartenance -0.349718
nb_devoirs_total      0.021070
age                  0.022501
distance_domicile_km 0.033332
heures_travail_remunere_sem 0.051151
retards_rendus       0.403059
abandon              1.000000
Name: abandon, dtype: float64

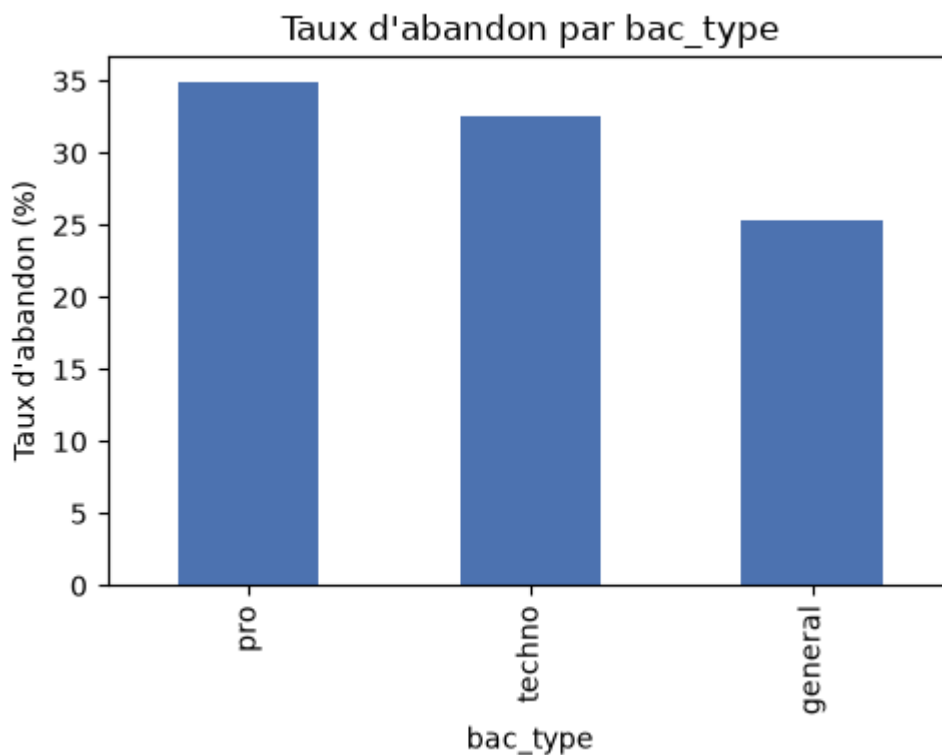
```

Interprétation. En excluant `moyenne_finale` (l'autre cible, pas une feature), les corrélations les plus fortes avec `abandon` sont toutes des variables d'engagement mesurables a mi-S1 :

`taux_presence_pct` (-0.58), `connexions_lms_30j` (-0.57), `ressources_consultees` (-0.54), `motivation` (-0.52), `heures_lms_total` (-0.50), `messages_forum` (-0.47) et `retards_rendus` (+0.40, seule corrélation positive marquée). A l'inverse, `age`, `distance_domicile_km`,

`heures_travail_remunere_sem` et `nb_devoirs_total` sont quasiment non corrélés (< 0.05 en valeur absolue) : ce sont probablement des prédicteurs faibles, à confirmer avec l'importance des variables une fois le modèle entraîné.

6.4 Taux d'abandon par type de bac



```
bac_type
pro      34.93
techno   32.55
general  25.38
Name: abandon, dtype: float64
```

Interprétation. Le type de bac est discriminant : 34.9 % d'abandon pour un bac professionnel, 32.6 % pour un bac technologique, contre seulement 25.4 % pour un bac général. Un écart de près de 10 points entre `pro` et `general` en fait un candidat feature solide, à encoder en nominal (one-hot) puisqu'il n'y a pas d'ordre naturel entre les 3 filières de bac.

7. Préparation des données (nettoyage, manquants, transformations, features)

7.1 Colonnes non pertinentes pour la modélisation -> dataframe silver

Comme indiqué au chapitre les colonnes suivantes sont supprimées : Suppression des identifiants (RGPD), variables leurres, colonnes à fuite temporelle, ainsi que `sexe` et `boursier`. Résultat enregistré dans un dataframe `df_silver` distinct (le `df_bronze` original n'est pas modifié). Les

cibles `abandon` et `moyenne_finale` sont conservees (cette dernière ne devra jamais servir de variable explicative pour predire `abandon`).

(5240, 26)

7.2 Homogénéisation de la colonne `date_inscription` (dataframe `silver`)

Formats melanges dans le CSV source (`AAAA-MM-JJ` , `JJ/MM/AAAA` , `JJ Mon AAAA`). Parsing unifie (jour en premier) puis reecriture au format `JJ-MM-AAAA` , applique uniquement sur `df_silver` (le `df_bronze` original n'est pas modifie).

```
5207    06-09-2024
3321    05-09-2024
280     18-09-2024
4971    06-09-2024
4       04-09-2024
3075    11-09-2024
1247    20-09-2024
200     27-09-2024
4128    18-09-2024
1179    03-09-2024
Name: date_inscription, dtype: object
```

7.3 Conversion des colonnes numériques mal typees

Les colonnes `distance_domicile_km` et `taux_presence_pct` sont en `object` a cause d'un melange de virgule decimale et d'unites (`km` , `%`) heritees du CSV source.

```

Doublons avant suppression : 40 (0.76 %)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5200 entries, 0 to 5199
Data columns (total 26 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   filiere                               5200 non-null   object
1   age                                   5200 non-null   int64
2   bac_type                             5200 non-null   object
3   mention_bac                         4992 non-null   object
4   etablissement_origine               4940 non-null   object
5   distance_domicile_km              4888 non-null   float64
6   heures_travail_remunere_sem        4576 non-null   float64
7   taux_presence_pct                  5200 non-null   float64
8   connexions_lms_30j                 4992 non-null   float64
9   heures_lms_total                   5200 non-null   float64
10  ressources_consultees               5200 non-null   int64
11  retards_rendus                     5200 non-null   int64
12  nb_devoirs_total                   5200 non-null   int64
13  nb_devoirs_rendus                  5200 non-null   int64
14  messages_forum                     5200 non-null   int64
15  motivation                         4940 non-null   float64
16  satisfaction                       4784 non-null   float64
17  sentiment_appartenance              4836 non-null   float64
18  date_inscription                   5200 non-null   object
19  moyenne_finale                     5200 non-null   float64
20  abandon                            5200 non-null   int64
21  faculte                            5200 non-null   object
22  ects_semestre                      5200 non-null   int64
23  capacite_accueil                   5200 non-null   int64
24  volume_horaire_s1                  5200 non-null   int64
25  taux_reussite_historique_pct        5200 non-null   int64
dtypes: float64(9), int64(11), object(6)
memory usage: 1.0+ MB

```

7.4 Transformation des données

7.5 Homogénéisation de la colonne bac_type (dataframe silver)

Variantes de casse/accents/abreviations (GEN , general , Generale , General , general accentue , TECHNO , Technologique , techno , Professionnel , pro , PRO , " pro").
Normalisation vers 3 catégories : general , techno , pro .

```

bac_type
general    3197
techno     1402
pro         601
Name: count, dtype: int64

```

7.6 Homogénéisation de la colonne filière (dataframe silver)

Mêmes variantes de casse/espaces que pour la jointure bronze. Normalisation vers la casse canonique du catalogue des formations (`Biologie` , `Droit` , `Gestion` , `Informatique` , `Lettres` , `Mathematiques` , `Psychologie` , `STAPS`).

```
filiere
Informatique      858
Gestion           819
Droit             730
Psychologie       713
Biologie          621
STAPS            550
Mathematiques     485
Lettres           424
Name: count, dtype: int64
```

7.7 Homogénéisation de la colonne mention_bac (dataframe silver)

Abreviations et casse melangees (`P` / `passable` / `Passable` , `AB` / `ab` / `Assez Bien` , `B` / `Bien` , `TB` / `tres bien` / `Tres Bien`). Normalisation vers 4 catégories : `Passable` , `Assez Bien` , `Bien` , `Tres Bien` .

Note : `etablissement_origine` ne contient aucune variante (`lycee_public` , `lycee_prive` , `cfa` , `autre`) et ne nécessite pas d'homogénéisation.

```
mention_bac
Passable      1952
Assez Bien    1602
Bien          995
Tres Bien     443
NaN           208
Name: count, dtype: int64
```

7.8 Taux d'abandon global et par filière

La classe de détection n'est ni majoritaire, ni trop minoritaire. L'ajout de données "artificielle" (SMOTE) pour un ré-équilibrage n'est pas nécessaire.

Taux d'abandon global : 28.44 %

```
filiere
Gestion      31.14
Droit        29.32
STAPS        29.09
Lettres      28.07
Mathematiques 28.04
Biologie     28.02
Informatique 27.74
Psychologie  25.67
Name: abandon, dtype: float64
```

7.9 Encodage des variables catégorielles (dataframe silver)

L'encodage est nécessaire surtout si on utilise une régression logistique dans la comparaison de modèles (mentionnée dans le résumé exécutif) : un modèle linéaire ne peut pas interpréter des chaînes de caractères, et un simple encodage ordinal (0,1,2...) imposerait un ordre arbitraire et fictif entre les filières. Pour les modèles à arbres (RandomForest, boosting), c'est moins critique mais reste standard pour comparer les familles de modèles sur la même matrice de caractéristiques.

Application à :

- **mention_bac** (ordinaire, Passable < Assez Bien < Bien < Tres Bien) : encodage ordinal {0, 1, 2, 3}. Les valeurs manquantes restent NaN (l'imputation réelle se fera plus tard, après le split train/test, pour éviter toute fuite).
- **filiere**, **bac_type**, **faculte** (nominales, sans valeur manquante) : one-hot encoding classique (`pd.get_dummies`), sans colonne `_nan` puisqu'aucune de ces colonnes ne contient de valeur manquante.
- **etablissement_origine** (nominale, ~5 % de manquants) : one-hot encoding avec `dummy_na=True`, ce qui crée une colonne indicatrice `etablissement_origine_nan` explicite. Cette colonne n'est pas une imputation (aucune valeur n'est devinée) : elle rend simplement la non-réponse visible comme un signal potentiel, en cohérence avec le choix fait de reporter toute imputation statistique après le split.

(5200, 44)

7.10 Bilan des valeurs nulles par colonne (dataframe silver)

	n_nulls	pct_nulls
heures_travail_remunere_sem	624	12.0
satisfaction	416	8.0
sentiment_appartenance	364	7.0
distance_domicile_km	312	6.0
motivation	260	5.0
mention_bac	208	4.0
connexions_lms_30j	208	4.0

La proportion de valeurs nulles n'est pas trop importante. On va pouvoir utiliser des méthodes de remplissage afin d'obtenir un dataframe sans aucune valeurs nulles.

7.11 Stratégie d'imputation des valeurs nulles (dataframe silver)

heures_travail_remunere_sem et connexions_lms_30j contiennent déjà beaucoup de vrais 0 parmi les valeurs non nulles : un NaN représente donc une non-réponse réelle, pas un 0 implicite. Cela écarte une imputation par 0.

Colonne	Type	Méthode envisagée	Pourquoi
heures_travail_remunere_sem	numérique continue	médiane	distribution asymétrique (beaucoup de 0, quelques valeurs hautes) ; médiane robuste aux outliers
connexions_lms_30j	numérique discrète	médiane	idem, comptage non symétrique
distance_domicile_km	numérique continue	médiane	pas de lien attendu avec le décrochage, imputation simple suffit
satisfaction , sentiment_appartenance , motivation	ordinales (1-5, auto-évaluation)	médiane + indicateur binaire _manquant	la non-réponse à ces questionnaires d'engagement peut elle-même être corrélée au décrochage ; le flag conserve ce signal plutôt que de l'imputer en silence
etablissement_origine	catégorielle	nouvelle catégorie "Inconnu"	évite affecter artificiellement la donnée majoritaire (lycee_public) sur des cas où on ne sait rien
mention_bac	catégorielle ordinale	nouvelle catégorie "Inconnue"	même logique, cohérent avec les autres catégories déjà nettoyées

Point d'attention (anti-fuite) : l'imputation (médiane) devra être calculée uniquement sur le train (jeux d'entraînement) puis appliquée au test, pas sur l'ensemble du dataset en une seule fois.

7.12 Dataframe gold

7.12.1 Feature engineering : taux de rendu des devoirs

Calcul du `taux_rendu = nb_devoirs_rendus / nb_devoirs_total`. Si la valeur maximum est de 1 cela veut dire que la colonne `nb_devoirs_rendus` est renseignée en fin de semestre, elle est donc à supprimer pour éviter une fuite de données. Création de `df_gold` à partir de `df_silver` (celui-ci reste inchangé). Ajout de `taux_rendu = nb_devoirs_rendus / nb_devoirs_total`, puis suppression des deux colonnes sources (redondantes une fois le ratio calculé).

7.12.2 Vérification : maximum du taux de rendu

maximum du taux de rendu : 1.0

Si le ratio max = 1, on n'ajoute pas de colonne pour le taux de ration et on supprime la colonne `nb_devoirs_rendus` pour éviter les fuites de données. On crée ensuite le gold dataset.

Comme indiqué plus haut, le gold dataset sera imputer pour remplacer les NaN par des valeurs cohérentes divisées entre le jeu d'entraînement (train) et à part le jeu de test.

fichier gold créé : data\gold\gold-dataset.csv

7.13 Split train/test et imputation

7.13.1 Split puis imputation (fit sur le train uniquement)

Comme discuté précédemment, l'imputation des 7 colonnes numériques restant à NaN (`heures_travail_remunere_sem`, `satisfaction`, `sentiment_appartenance`, `distance_domicile_km`, `motivation`, `mention_bac`, `connexions_lms_30j`) doit être calculée uniquement sur le train pour éviter toute fuite train/test. Le split est donc fait avant, stratifié sur `abandon` (cible principale), puis `SimpleImputer(strategy="median")` est `fit` sur `df_gold_train` et appliqué (`transform`) au train et au test avec les mêmes valeurs.

train: (4160, 43) - nulls: 0
test: (1040, 43) - nulls: 0

8. Choix du modèle et démarche scientifique (baseline, modèles, comparaison)

Comme nous l'avons vu jusque là, c'est un problème de prédiction par classification.

8.1 Comparaison de plusieurs modèles (classification abandon)

Plutôt qu'un choix figé a priori, comparaison de 3 familles évoquées dans le résumé exécutif :

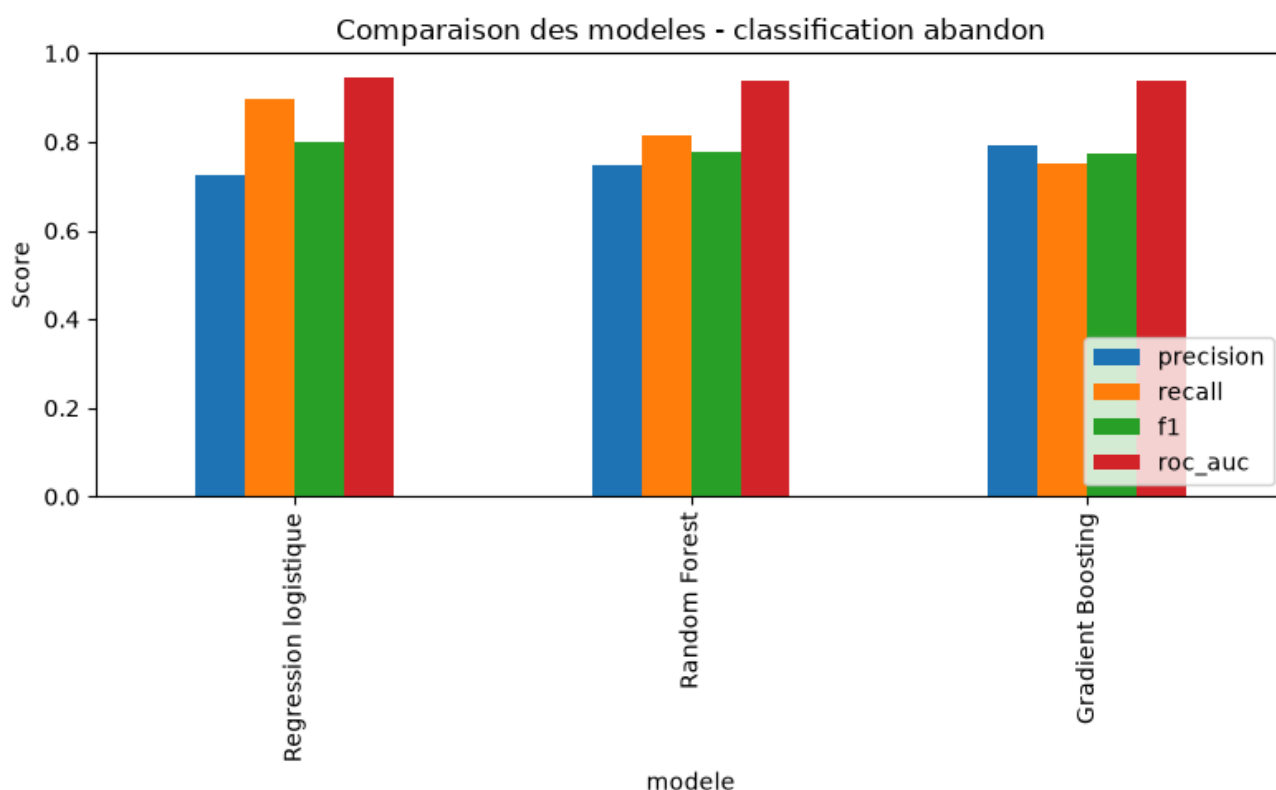
régression logistique (baseline interprétable, avec mise à l'échelle des variables), **Random Forest** et **Gradient Boosting** (arbres, pas de mise à l'échelle nécessaire). `class_weight="balanced"`

compense le déséquilibre de la cible (28 % d'abandon) pour la logistique et la Random Forest.

`date_inscription` est exclue (texte, pas encore transformée en feature numérique) ainsi que

`moyenne_finale` (fuite : c'est l'autre cible).

	accuracy	precision	recall	f1	roc_auc
modele					
Regression logistique	0.873	0.724	0.895	0.801	0.946
Random Forest	0.868	0.746	0.814	0.779	0.938
Gradient Boosting	0.873	0.791	0.753	0.772	0.939

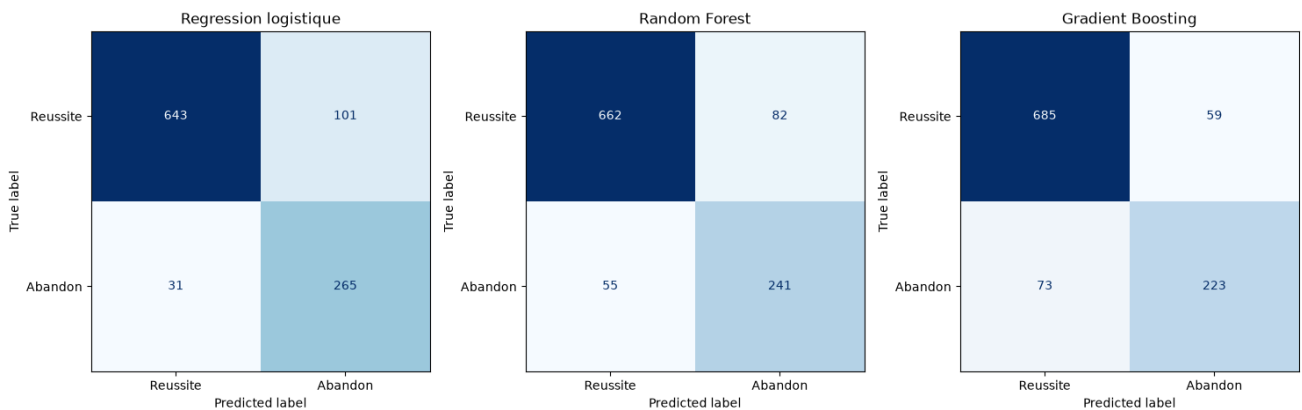


Interprétation. Sur ce jeu de données, la **régression logistique** obtient le meilleur rappel (0.895) et le meilleur ROC-AUC (0.946), devant Random Forest (rappel 0.814, AUC 0.938) et Gradient Boosting (rappel 0.753, AUC 0.939). C'est cohérent avec l'analyse de corrélation du chapitre 6 : les variables les plus prédictives (`taux_presence_pct` , `connexions_lms_30j` , `ressources_consultees` ...) ont une relation globalement linéaire avec `abandon` , ce qui avantage un modèle linéaire face à des

arbres qui n'apportent pas d'interactions non-linéaires supplémentaires ici. Étant donné la priorité donnée à la réduction des faux négatifs (rappel), la régression logistique est pour l'instant le meilleur candidat, avec l'avantage supplémentaire d'être nativement interprétable (coefficients) plutôt que de nécessiter SHAP.

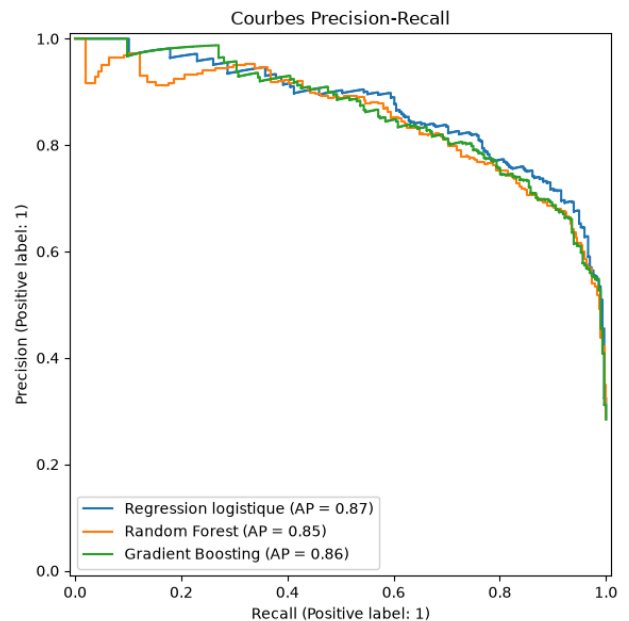
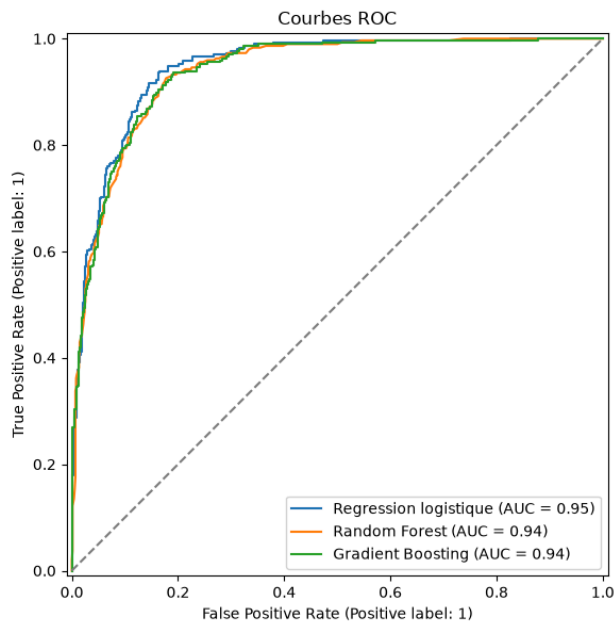
Cette comparaison reste à affiner : validation croisée plutôt qu'un seul split, recherche d'hyperparamètres (`GridSearchCV` / `RandomizedSearchCV`), et ajustement du seuil de décision (actuellementment 0.5) en fonction du coût métier d'un faux négatif discuté au chapitre 2.

8.1.1 Matrices de confusion des 3 modèles



Interprétation. La régression logistique fait le moins de faux négatifs (31 étudiants à risque non détectés, contre 55 pour Random Forest et 73 pour Gradient Boosting sur 1040 lignes de test), cohérent avec son meilleur rappel (0.895). En contrepartie elle génère plus de faux positifs (101, contre 82 et 59), donc plus d'étudiants signalés à tort. C'est l'arbitrage recall/précision déjà discuté au chapitre 2 : étant donné la priorité donnée au coût d'un faux négatif, ce compromis va dans le sens souhaité pour la régression logistique.

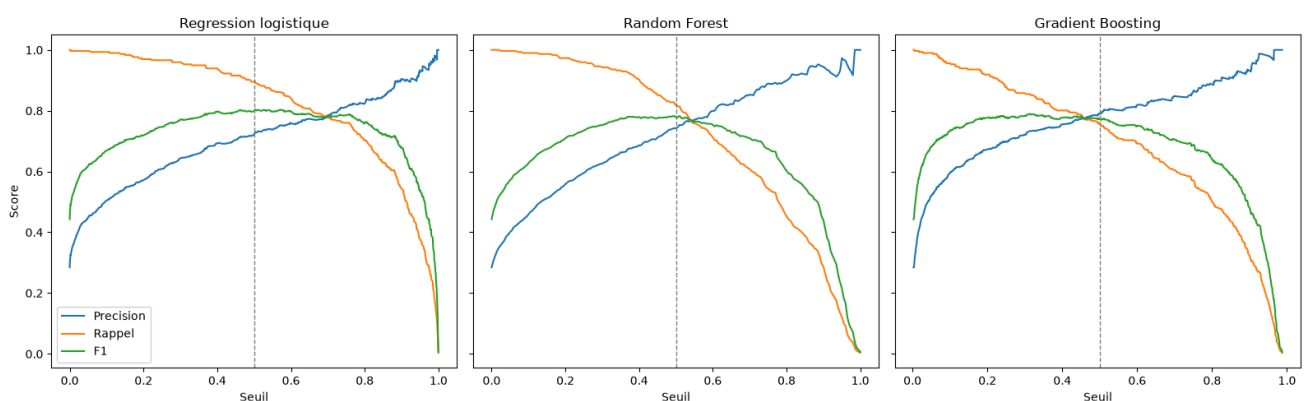
8.1.2 Courbes ROC et Précision-Recall des 3 modèles



Interprétation. La courbe ROC confirme l'AUC : la régression logistique (0.946) domine légèrement Gradient Boosting (0.939) et Random Forest (0.938) sur presque toute la plage de seuils, mais l'écart reste faible. La courbe Précision-Recall est plus parlante ici compte tenu du déséquilibre de la cible (28 % d'abandon) : elle montre le compromis réel entre détecter plus d'abandons (rappel) et limiter les fausses alertes (précision), et confirme que la régression logistique conserve un meilleur rappel à précision égale sur la majorité de la courbe. C'est cette courbe, plus que le ROC, qui devrait guider le choix final du seuil de décision en fonction du coût métier d'un faux négatif.

8.1.3 Courbe seuil-vs-métriques (choix du seuil de décision)

Précision, rappel et F1 traces en fonction du seuil de décision (au lieu du seuil implicite de 0.5 utilisé jusqu'ici), pour les 3 modèles. Objectif : visualiser le compromis réel avant de fixer un seuil de production, en cohérence avec la priorité donnée au coût d'un faux négatif.



Seuil qui maximise le F1 : 0.503 (F1 = 0.804)

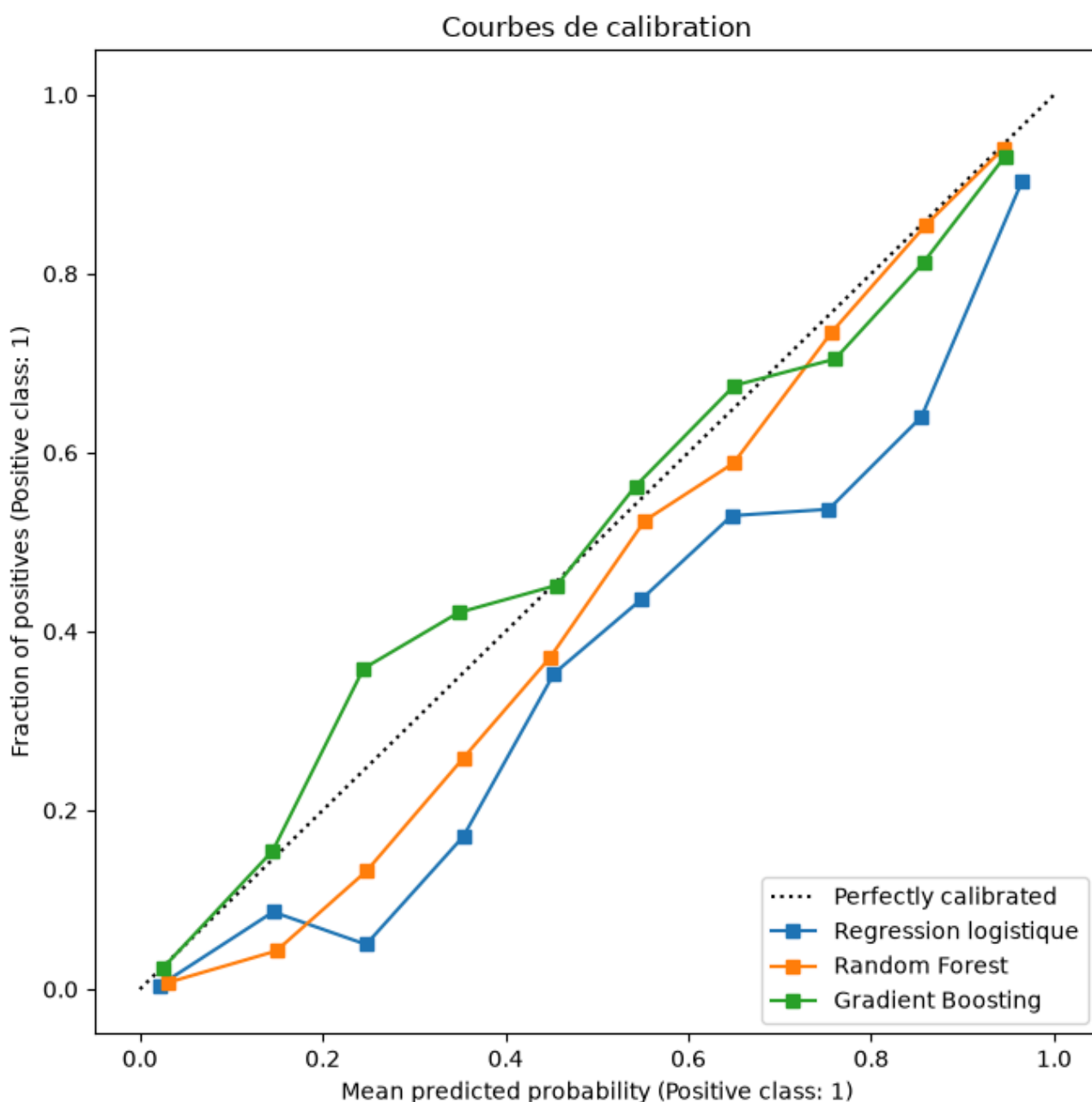
Seuil pour un rappel ≥ 0.95 : 0.329 (precision correspondante = 0.653)

Interprétation. Pour la régression logistique, le seuil qui maximise le F1 (0.503) est quasiment identique au seuil par défaut (0.5) : les métriques déjà calculées plus haut n'étaient donc pas défavorisées par un choix de seuil arbitraire. Mais si l'objectif est de pousser le rappel à 0.95 (rater seulement 5 % des abandons réels, cohérent avec la priorité donnée au coût d'un faux négatif), il faut

baisser le seuil a environ **0.33** : la précision tombe alors a ~ 0.65 , donc davantage de fausses alertes (plus de tutorat déclenché pour des étudiants qui n'en avaient pas besoin). Le choix final entre ces deux seuils (ou une valeur intermediaire) devrait être validé avec la Direction de la réussite étudiante, qui est la seule a pouvoir chiffrer le coût réel d'un faux négatif vs celui d'un faux positif.

8.1.4 Courbes de calibration

Vérifié si la probabilité prédite (`predict_proba`) reflète la vraie fréquence observée : utile car le choix du seuil de décision, discuté au chapitre précédent, ne veut dire quelque chose que si les probabilités sont fiables.



Regression logistique - Brier score : 0.0935

Random Forest - Brier score : 0.0930

Gradient Boosting - Brier score : 0.0898

Interprétation. Les 3 modèles sont assez proches en calibration (Brier score entre 0.090 et 0.094, plus bas = meilleur), Gradient Boosting légèrement devant. Point d'attention pour la régression logistique

et Random Forest : `class_weight="balanced"` a été utilisé pour compenser le déséquilibre de la cible et améliorer le rappel, mais cela déforme mécaniquement les probabilités prédites (elles ne reflètent plus le vrai taux de base de 28.4 % d'abandon, seulement un ordre relatif de risque). Conséquence pratique : les seuils calculés au chapitre précédent (0.503 pour le F1, 0.33 pour un rappel de 0.95) sont valides *pour ce modèle pondéré*, mais ne seraient plus les bons si on entraînait sans `class_weight="balanced"` ou si on recalibrerait les probabilités (`CalibratedClassifierCV`). A garder en tête avant de figer un seuil de production.

9. Entraînement, validation et ajustement (sélection du modèle final)

9.1 Validation croisée et optimisation des hyperparamètres

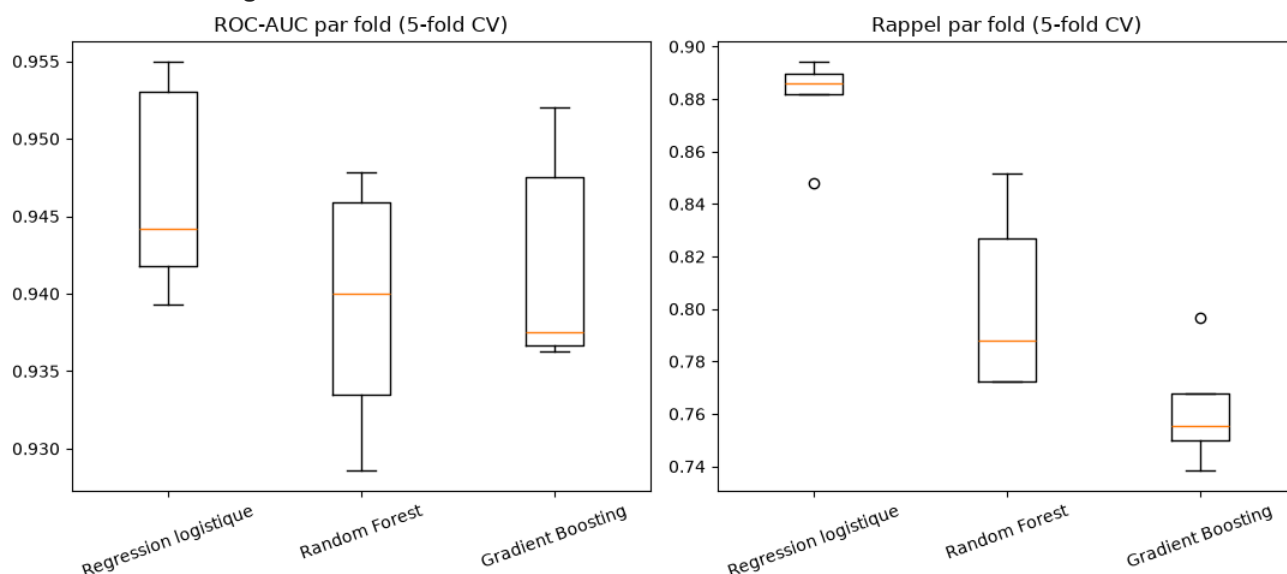
Avant de calculer l'importance des variables (qui n'a de sens que sur un modèle stable et *tune*), deux étapes : (1) vérifier que l'écart observé plus haut entre les 3 modèles n'est pas un artefact du seul split train/test utilisé jusqu'ici, via une validation croisée 5-fold sur le train ; (2) optimiser les hyperparamètres de chaque modèle (recherche sur `roc_auc`, métrique la plus robuste au déséquilibre de la cible) avant de re-évaluer sur le test.

9.1.1 Stabilité par validation croisée (modèles par défaut)

Regression logistique: AUC=0.947 +/- 0.006 Recall=0.880 +/- 0.016

Random Forest: AUC=0.939 +/- 0.007 Recall=0.802 +/- 0.032

Gradient Boosting: AUC=0.942 +/- 0.007 Recall=0.762 +/- 0.020



Interprétation. Les écarts-types sont faibles (AUC : +/-0.006 à +/-0.007 ; rappel : +/-0.016 à +/-0.032 selon le modèle) et les boîtes ne se superposent pas : l'avance de la régression logistique sur le rappel n'est donc pas un artefact du split choisi (`random_state=42`), elle se confirme sur les 5 folds.

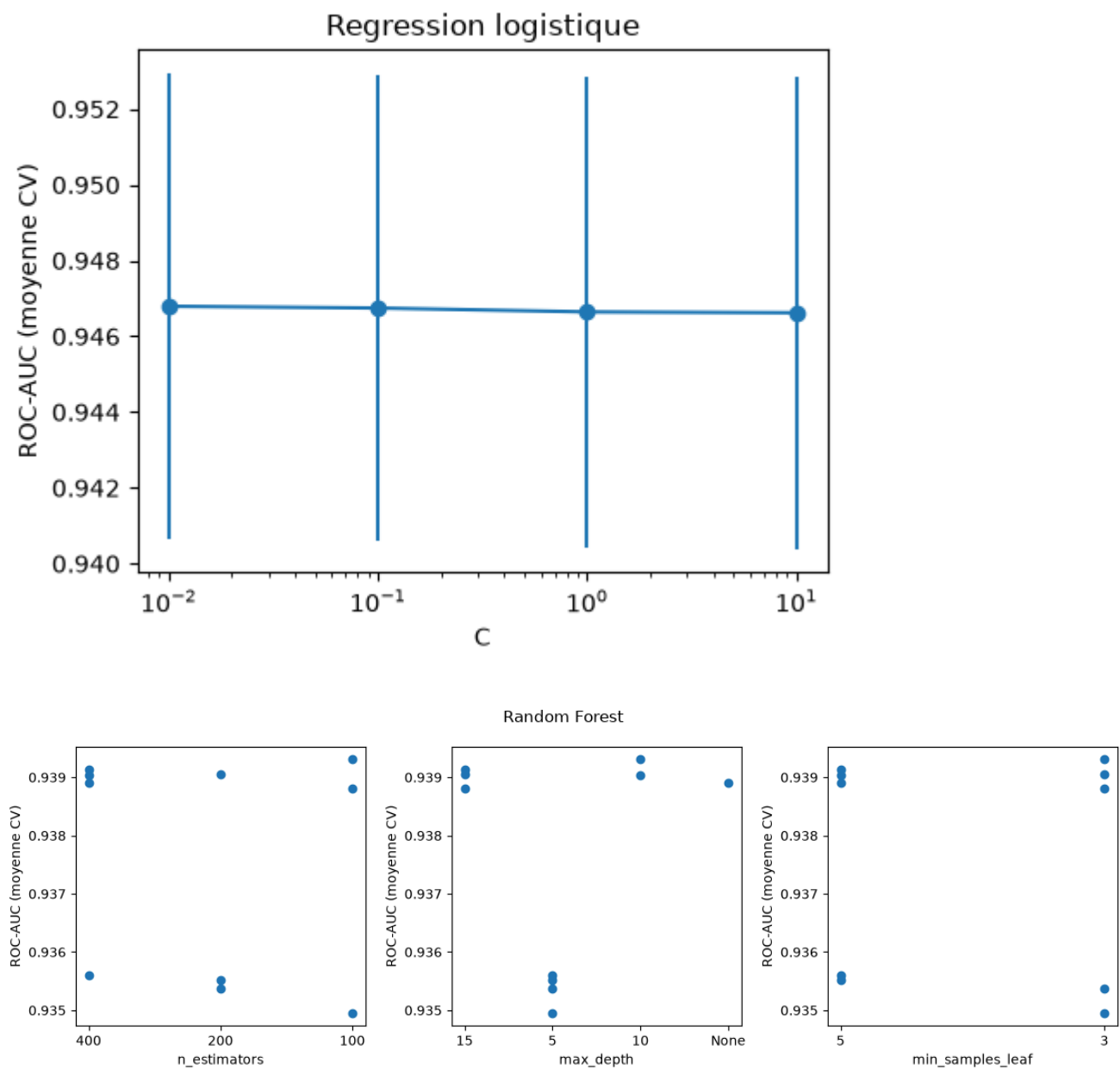
9.1.2 Optimisation des hyperparamètres (recherche sur ROC-AUC)

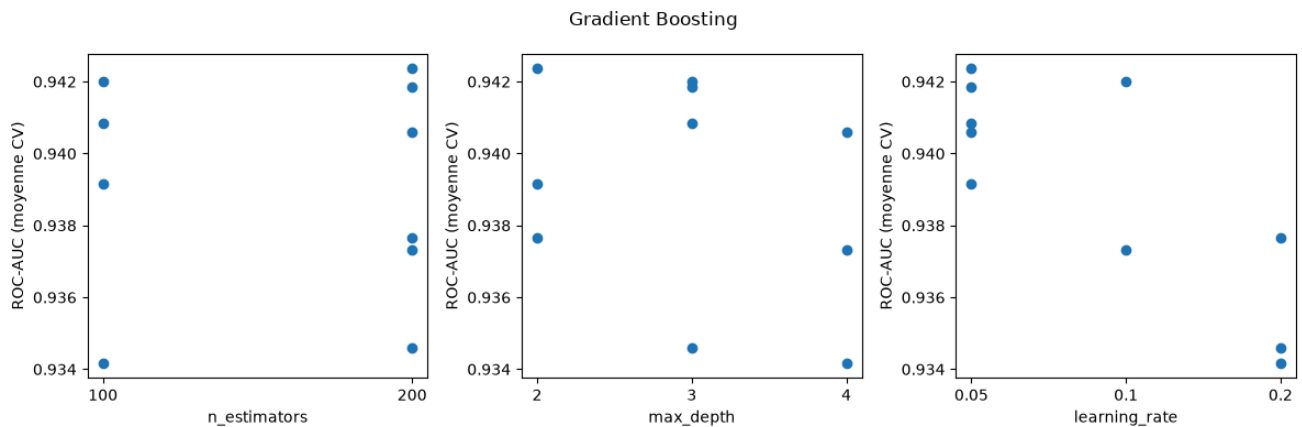
Regression logistique - meilleurs parametres : {'clf__C': 0.01} (AUC cv = 0.947)

Random Forest - meilleurs parametres : {'n_estimators': 100, 'min_samples_leaf': 3, 'max_depth': 10} (AUC cv = 0.939)

Gradient Boosting - meilleurs parametres : {'n_estimators': 200, 'max_depth': 2, 'learning_rate': 0.05} (AUC cv = 0.942)

9.1.3 Courbes d'optimisation des hyperparamètres





Interprétation. Pour la régression logistique, le score varie à peine avec C (0.9468 à 0.9466 entre $C=0.01$ et $C=10$) : le modèle est très peu sensible à la régularisation ici, $C=0.01$ (le plus régularise) gagne de très peu. Pour Random Forest et Gradient Boosting, chaque hyperparamètre pris isolément montre un effet marginal limité sur le score CV (nuage de points assez plat pour chacun) : aucun des deux modèles à base d'arbres ne bénéficie d'un réglage particulièrement sensible sur cette grille, ce qui confirme que l'écart de performance avec la logistique vient surtout du choix de la famille de modèle (cf. 7.6), pas d'un manque de tuning.

9.1.4 Re-évaluation des modèles tunes sur le test

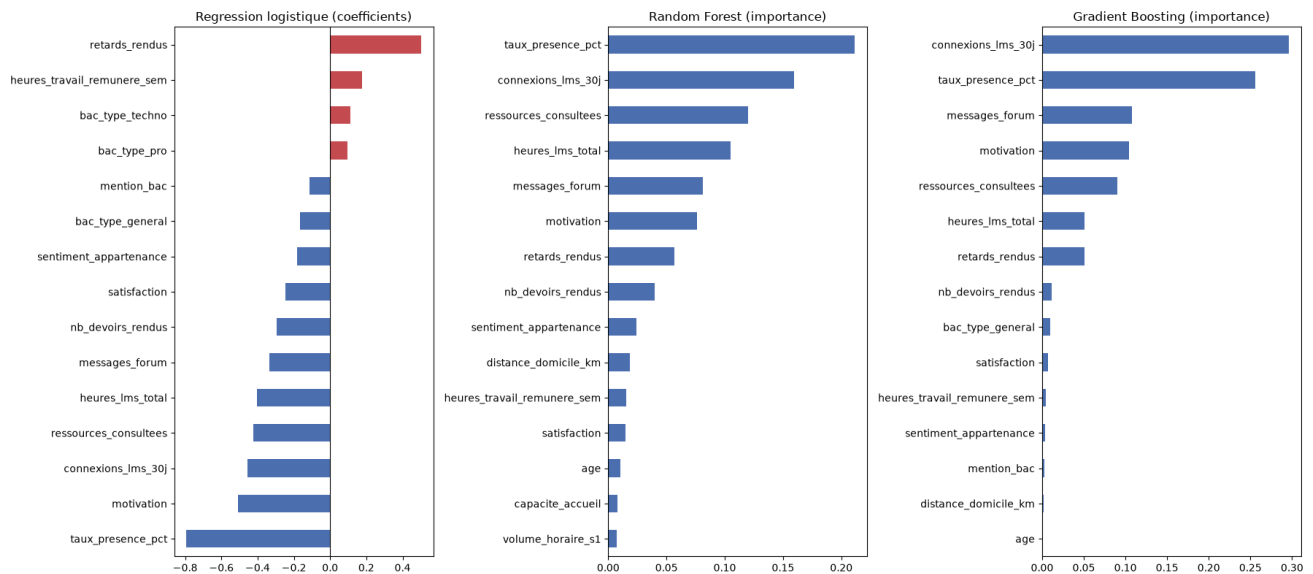
	precision	recall	f1	roc_auc
modele				
Regression logistique	0.729	0.899	0.805	0.946
Random Forest	0.725	0.865	0.789	0.941
Gradient Boosting	0.796	0.740	0.767	0.941

Interprétation. Après tuning, le classement ne change pas : la régression logistique reste devant (recall 0.899, AUC 0.946 sur le test), avec un $C=0.01$ optimal (forte régularisation), cohérent avec la présence de variables corrélées entre elles identifiée au chapitre 6. Random Forest et Gradient Boosting s'améliorent légèrement par rapport à leurs versions par défaut mais ne rattrapent pas la logistique. Conclusion : le choix de la régression logistique (tunée) est confirmé, et ce n'est plus basé sur un seul split mais sur une validation croisée stable. Les étapes suivantes (importance des variables, seuil final) peuvent maintenant s'appuyer sur `modeles_tunes["Regression logistique"]` plutôt que sur la version par défaut.

9.2 Importance des variables / coefficients (modèles tunes)

Calculée maintenant que les modèles sont tunes et valides par validation croisée (chapitre précédent), pour vérifier l'explicabilité et la cohérence avec les corrélations trouvées en EDA (7.6). Pour la régression logistique : coefficients signes (positif = augmente le risque d'abandon, négatif = le

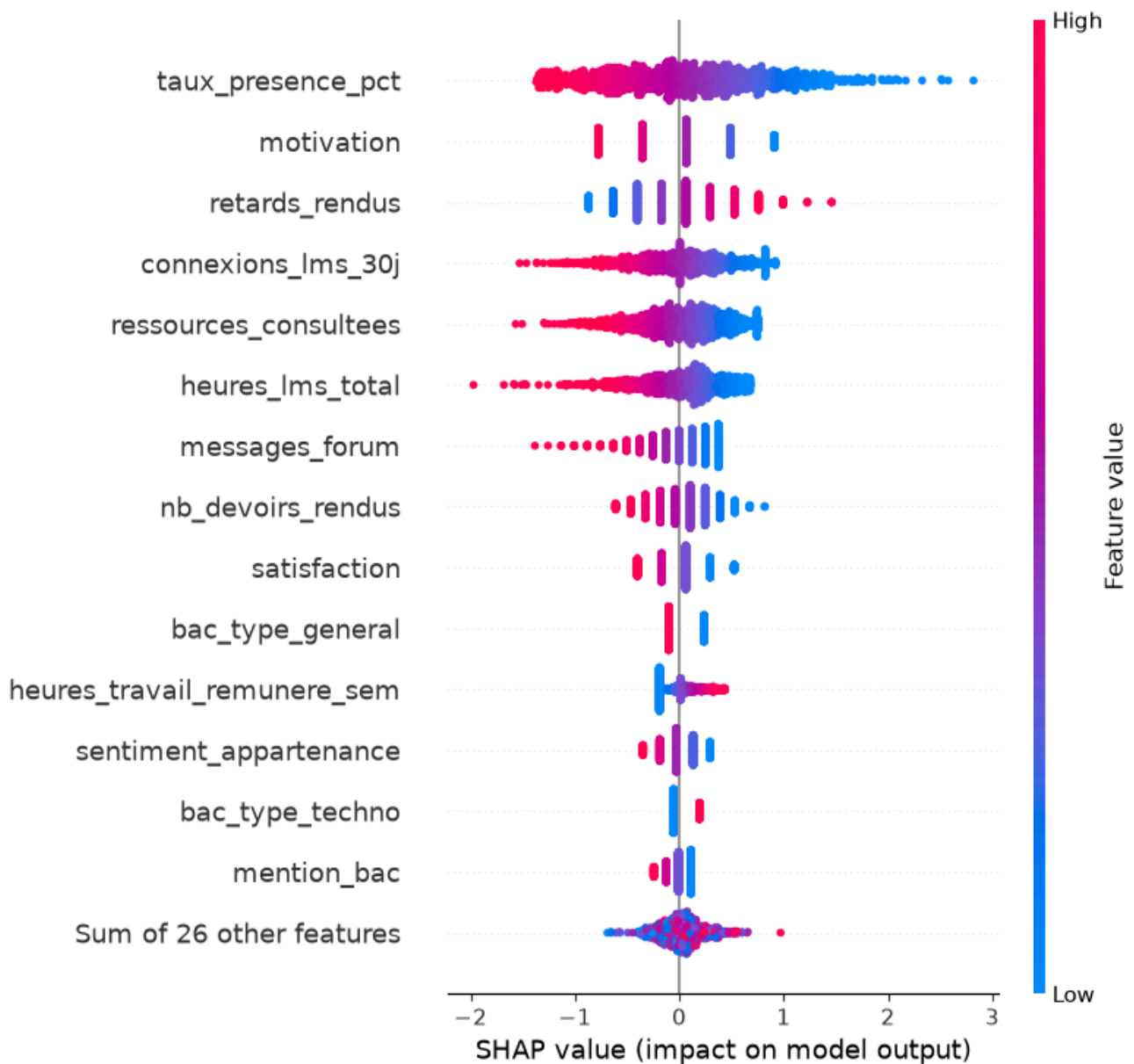
diminue), lisibles directement car les features sont standardisées. Pour Random Forest et Gradient Boosting : importance par réduction d'impurité (sans signe).



Interprétation. Les 3 modèles s'accordent sur l'essentiel : **taux_presence_pct**, **connexion_lms_30j**, **ressources_consultees**, **motivation**, **heures_lms_total** et **messages_forum** figurent dans le top des 3 classements. C'est exactement les variables d'engagement identifiées comme les plus corrélées à **abandon** en EDA (7.6), ce qui renforce la confiance dans le modèle (pas d'appui sur une variable inattendue ou une fuite résiduelle). **retards_rendus** a un coefficient positif marqué (+0.50) pour la logistique : c'est la seule variable qui *augmente* le risque, cohérent avec le signe positif déjà observé en corrélation (+0.40). Aucune variable administrative ou leurre (**age**, **distance_domicile_km**, **faculte_***, **filiere_***) n'apparaît dans le top 15 d'aucun des 3 modèles : elles peuvent être présentées comme peu utiles au modèle, ce qui simplifie le message aux équipes pédagogiques (l'engagement compte, pas le profil socio-démographique).

Sur le plan de l'explicabilité (exigence du cadrage métier, chapitre 4) : les coefficients de la régression logistique retenue peuvent déjà être présentés directement aux enseignants ("la présence en cours et l'activité sur le LMS sont les signaux les plus forts"). Le diagramme SHAP ci-dessous confirme ce classement et ajoute une explication instance par instance (utile pour justifier une alerte sur un étudiant précis, pas seulement au niveau global du modèle).

9.2.1 Diagramme SHAP (régression logistique retenue)



Interprétation. Le classement SHAP (importance moyenne absolue) confirme celui des coefficients : `taux_presence_pct` domine largement, suivie de `motivation`, `retards_rendus`, `connexions_lms_30j`, `ressources_consultees`, `heures_lms_total`, `messages_forum` puis `satisfaction`. Le diagramme beeswarm ajoute une information que le coefficient seul ne donne pas : pour `taux_presence_pct`, les points rouges (valeurs élevées de la variable) sont concentrés à gauche (SHAP négatif, risque plus faible) et les points bleus (valeurs faibles) à droite (SHAP positif, risque plus élevé), de façon très nette et sans inversion - la relation est bien monotone et homogène sur toute la plage de valeurs, ce qui justifie a posteriori l'usage d'un modèle linéaire plutôt que des arbres pour ce jeu de données.

En pratique, ce diagramme permettrait d'expliquer une prédiction individuelle à un enseignant ("cet étudiant est signalé à risque principalement à cause de sa faible présence et de ses retards de rendu") plutôt que de se limiter à une explication globale du modèle.

9.3 Modèle retenu : régression logistique

La régression logistique tunee (`C=0.01` , `class_weight="balanced"`) est le modèle retenu.

Pourquoi elle l'emporte sur les 3 criteres qui comptent ici :

- **Performance** : meilleur rappel (0.899) et meilleur AUC (0.946 test, 0.947 en CV), confirmé sur 5 folds donc pas un artefact de split, et confirmé après tuning des 3 modèles.
- **Alignement métier** : le rappel le plus élevé = moins de faux négatifs (31 vs 55-73 pour RF/GB) - exactement la priorité fixée au chapitre 2 (coût d'un faux négatif > coût d'un faux positif).
- **Explicabilité** : coefficients directement lisibles et presentables aux équipes pédagogiques (5.4), sans nécessiter SHAP, et cohérents avec l'EDA (les variables d'engagement dominant, aucune variable démographique/leurre en tête).

Le compromis assume : elle génère plus de faux positifs (précision 0.729 vs 0.796 pour Gradient Boosting) - plus d'étudiants signalés à tort, donc plus de tutorat déclenché pour des profils qui n'en avaient pas besoin. Vu la priorité donnée aux faux négatifs, ce compromis va dans le bon sens.

9.4 Suivi des configurations avec MLflow

Plutôt que de garder les comparaisons de modèles/hyperparamètres uniquement dans la sortie des cellules, chaque configuration testée (modèle par défaut, puis modèle tune) est journalisée comme un run MLflow : paramètres, métriques de test, et le modèle final (régression logistique tune) avec sa signature. Même convention que `scripts/demo_versioning.py` (backend SQLite, nécessaire pour le Model Registry).

```
<Experiment: artifact_location='file:C:/Formation IA/Examen/ProjetDecrochage/mlruns/1', creation_time=1785762626895, effective_trace_archival_retention=None, experiment_id='1', last_update_time=1785762626895, lifecycle_stage='active', name='decrochage-abandon-classification', tags={}, trace_location=None, workspace='default'>
```

9.4.1 Journalisation des configurations par défaut (baseline)

9.4.2 Journalisation des configurations tunees, et enregistrement du modèle retenu

run retenu : 1561360b4ba945c4a9d86856cd95b263

Interprétation. 6 runs sont journalisés (3 modèles x baseline/tune), comparables dans l'UI MLflow (`uv run mlflow ui --backend-store-uri sqlite:///mlflow.db`) plutôt que dans des print éparpillés dans le notebook. Chaque run garde la trace exacte des hyperparamètres et des métriques de test correspondantes, ce qui permet de reproduire ou de comparer une configuration précise sans réexécuter tout le notebook. Le modèle final (régression logistique tune) est en plus enregistré comme artefact MLflow avec sa signature d'entrée/sortie, et versionne au Model Registry sous le nom `decrochage-abandon-logistique` - la même démarche que celle utilisée par `scripts/demo_versioning.py` pour le modèle de maintenance prédictive, appliquée ici au modèle de décrochage.

Actions:

Démarrage de mlflow : `uv run mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000` puis accès via le navigateur sur <http://127.0.0.1:5000>.

9.5 Prédiction de moyenne finale

9.5.1 Préparation et comparaison de modèles

Deuxieme objectif du resume executif (2.1) : estimer `moyenne_finale` pour prioriser l'accompagnement. Modèle **totalelement indépendant** de la classification `abandon` : même jeu de features mi-semestre (`features` , déjà defini), même split `df_gold_train` / `df_gold_test` , mais `moyenne_finale` en cible et `abandon` exclue des features cette fois (fuite inverse). Pas de stacking entre les deux tâches (risque de fuite circulaire déjà signalé au chapitre 5) : deux modèles séparés, chacun evalue independamment.

Comparaison de 3 familles, en parallele de la classification : **Ridge** (baseline linéaire regularisee, avec mise a l'échelle), **Random Forest** et **Gradient Boosting** Regressor.

9.5.1.1 Qu'est-ce que Ridge ?

Ridge = régression linéaire classique, mais avec une pénalité qui empeche les coefficients de devenir trop grands (regularisation L2).

Pourquoi ce choix ici : une régression linéaire simple (sans pénalité) peut avoir des coefficients instables quand des variables sont corrélées entre elles — exactement notre cas (`filiere` / `bac_type` / `faculte` en one-hot, `taux_presence_pct` / `connexions_lms_30j` correles, cf. chapitre 6). Ridge rétrécit ces coefficients vers zero (sans jamais les annuler complètement, contrairement a Lasso), ce qui rend le modèle plus stable et moins sensible au bruit.

Le réglage clé : `alpha` — plus il est grand, plus la pénalité est forte (coefficients plus petits, modèle plus prudent). Testé plus bas via `GridSearchCV` : l'optimum trouve sera `alpha=50` (forte regularisation), cohérent avec ce qui avait déjà été observe pour la classification (`C=0.01` , forte regularisation aussi) — les deux modèles linéaires convergent vers "beaucoup regulariser" a cause des mêmes variables corrélées.

Ridge sert ici de **baseline linéaire**, pour comparaison avec Random Forest et Gradient Boosting.

	MAE	RMSE	R2
modele			
Gradient Boosting	2.250	3.048	0.692
Random Forest	2.315	3.144	0.672
Ridge	2.588	3.219	0.656

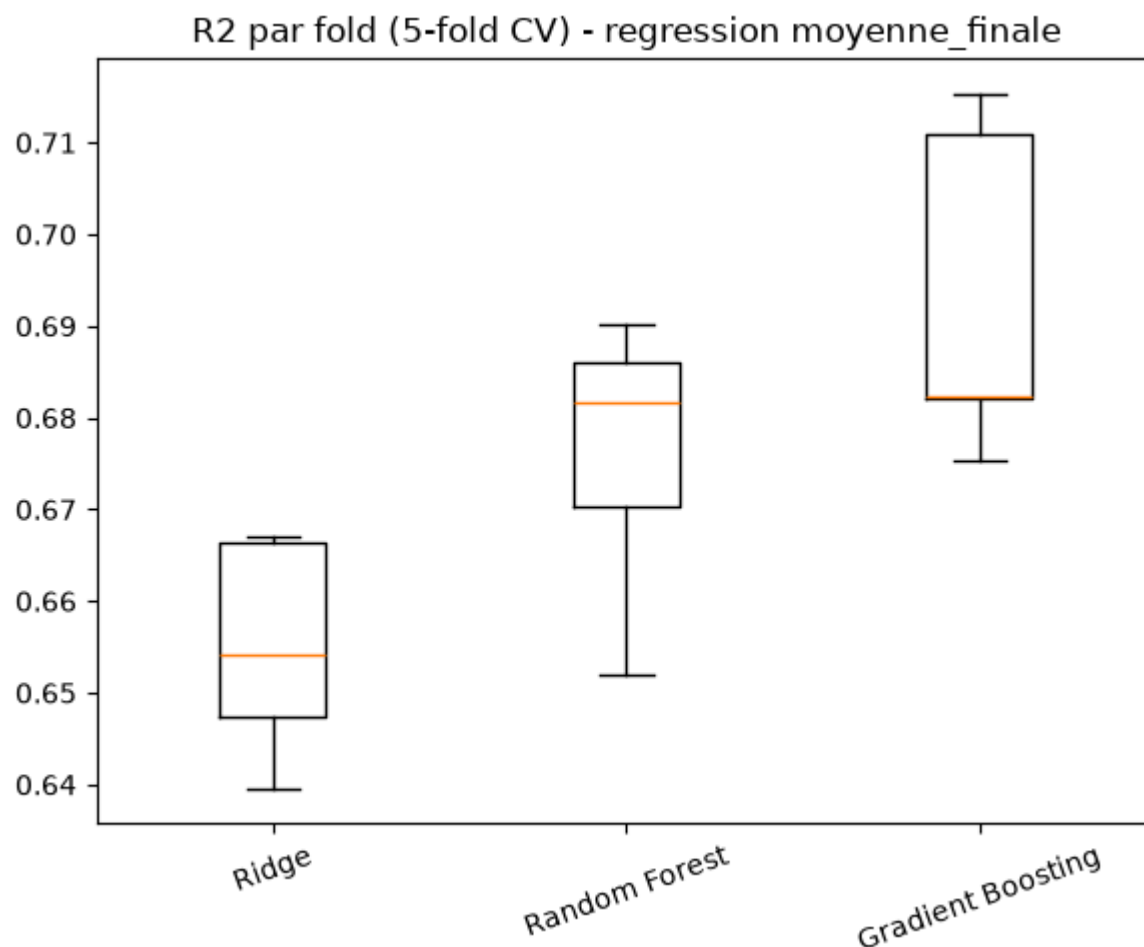
Interprétation. A l'inverse de la classification (ou la logistique gagnait), c'est ici **Gradient Boosting** qui devance les deux autres ($R^2=0.692$, $MAE=2.25$ points/20), suivi de Random Forest ($R^2=0.672$) puis Ridge ($R^2=0.656$, $MAE=2.59$). Les arbres captent mieux des interactions non linéaires pour estimer une note précise que pour la simple décision binaire d'abandon — cohérent avec le fait qu'une moyenne est une grandeur continue plus sensible aux combinaisons de facteurs (ex: faible motivation + bonne présence peut donner un résultat différent de faible motivation + faible présence, pas nécessairement linéaire).

9.5.2 Validation croisée et optimisation des hyperparamètres

Ridge: $R^2=0.655 \pm 0.011$ $MAE=2.580 \pm 0.066$

Random Forest: $R^2=0.676 \pm 0.014$ $MAE=2.299 \pm 0.045$

Gradient Boosting: $R^2=0.693 \pm 0.017$ $MAE=2.226 \pm 0.081$



Ridge - meilleurs paramètres : {'reg__alpha': 50.0} (R^2 cv = 0.655)

Random Forest - meilleurs paramètres : {'n_estimators': 200, 'min_samples_leaf': 3, 'max_depth': 15} (R^2 cv = 0.677)

Gradient Boosting - meilleurs paramètres : {'n_estimators': 100, 'max_depth': 3, 'learning_rate': 0.1} (R^2 cv = 0.693)

9.5.3 Re-évaluation des modèles tunes sur le test

	MAE	RMSE	R2
modele			
Gradient Boosting	2.250	3.048	0.692
Random Forest	2.299	3.136	0.674
Ridge	2.589	3.220	0.656

Interprétation. Le tuning ne change pas le classement : **Gradient Boosting** reste le meilleur modèle ($R^2=0.692$, MAE=2.25 points/20 sur le test), stable par rapport à la version par défaut — les hyperparamètres par défaut étaient déjà proches de l'optimum sur cette grille. Modèle retenu pour la régression : **Gradient Boosting Regressor** (`n_estimators=100`, `max_depth=3`, `learning_rate=0.1`).

Cette régression reste indépendante de la classification `abandon` (aucun partage de prédictions entre les deux modèles). Non branchée pour l'instant à la CLI/API (`decrochage predict` et `/predict-tabular` restent scopes sur la classification) — à ajouter si besoin d'exposer aussi l'estimation de `moyenne_finale` en production.

9.5.4 Modèle retenu pour la prédiction de moyenne finale

Gradient Boosting Regressor (`n_estimators=100`, `max_depth=3`, `learning_rate=0.1`).

Pourquoi ce modèle plutôt que Ridge ou Random Forest :

- **Performance** : meilleur R^2 (0.692) et meilleur MAE (2.25 points/20) sur le test, confirmé par validation croisée 5-fold ($R^2=0.693 \pm 0.017$) et stable après tuning (les hyperparamètres par défaut étaient déjà proches de l'optimum sur la grille testée).
- **Nature de la tâche** : à l'inverse de la classification `abandon` (ou la relation avec les variables d'engagement était globalement linéaire, cf. 7.6), estimer une note précise semble bénéficier d'interactions non linéaires entre facteurs (ex: faible motivation + bonne présence peut donner un résultat différent de faible motivation + faible présence) — un modèle à arbres capte mieux ce type d'effet qu'un modèle linéaire régularisé comme Ridge.
- **Cohérence avec le classifieur** : cette fois Random Forest et Gradient Boosting font mieux que le modèle linéaire (Ridge), à l'opposé de la classification — ce contraste renforce la lecture faite plus haut plutôt que de la contredire.

Le compromis assume : moins directement interprétable qu'une régression linéaire (coefficients), mais l'explicabilité est moins critique ici que pour `abandon` — cette régression sert à prioriser l'accompagnement (indicateur quantitatif), pas à justifier une décision individuelle sensible comme un signalement à risque.

9.5.5 Suivi des configurations avec MLflow (régression `moyenne_finale`)

Même démarche que pour la classification `abandon` (chapitre "Suivi des configurations avec MLflow") : chaque configuration testée (modèle par défaut, puis modèle tune) est journalisée comme un run MLflow, dans une expérience séparée (`decrochage-moyenne-finale-regression`) puisque c'est une tâche différente. Le modèle final (Gradient Boosting tune) est enregistré au Model Registry sous `decrochage-moyenne-finale-gb` .

```
<Experiment: artifact_location='file:C:/Formation IA/Examen/ProjetDecrochage/mlruns/2', creation_time=1785853787568, effective_trace_archival_retention=None, experiment_id='2', last_update_time=1785853787568, lifecycle_stage='active', name='decrochage-moyenne-finale-regression', tags={}, trace_location=None, workspace='default'>
```

9.5.5.1 Journalisation des configurations par défaut (baseline)

9.5.5.2 Journalisation des configurations tunees, et enregistrement du modèle retenu

run retenu : `ac907255bc53409996b2a73ecc051d25`

Interprétation. 6 runs supplémentaires sont journalisés (3 modèles x baseline/tune) dans l'expérience `decrochage-moyenne-finale-regression` , visibles séparément de la classification dans la même UI MLflow (`uv run mlflow ui --backend-store-uri sqlite:///mlflow.db`). Le modèle retenu (Gradient Boosting tune) est versionné au Model Registry sous `decrochage-moyenne-finale-gb` , distinct de `decrochage-abandon-logistique` — les deux modèles indépendants ont chacun leur propre historique de versions, comparable et reproductible sans repasser par le notebook.

9.6 Peut-on dériver l'abandon à partir de la note prédite ?

Alternative testée : plutôt que le classifieur logistique entraîné directement sur `abandon` , utiliser le score `-moyenne_finale_predite` (régresseur Gradient Boosting du chapitre précédent) comme indicateur de risque, puis seuiller. Pas de fuite : seule la note *prédite* (à partir des features mi-semester) est utilisée, jamais la vraie `moyenne_finale` . Le seuil est calibré par F1 maximal sur le train (même méthodologie que pour le classifieur, chapitre seuil-vs-métriques).

	AUC	precision	recall	f1
approche				
Classifieur logistique (direct)	0.946	0.729	0.899	0.805
Note prédite seuillée (~10.9/20)	0.939	0.744	0.804	0.773

Interprétation. Le classifieur direct reste meilleur sur tous les indicateurs sauf la précision (marginale) : AUC 0.946 vs 0.939, et surtout un rappel nettement supérieur (0.899 vs 0.804) — c'est le critère prioritaire ici (coût d'un faux négatif). Conclusion attendue : la régression optimise pour minimiser l'erreur sur une note continue, pas pour séparer les deux classes d'abandon, donc un classifieur entraîné directement sur cet objectif reste préférable. La régression sur `moyenne_finale`

garde sa valeur propre (estimer une note pour prioriser l'accompagnement), mais ne remplacé pas le classifieur pour la detection du risque d'abandon.

9.7 Fonction combinee : proba d'abandon + note prédite

Les deux modèles restent indépendants (entraînés séparément, chacun sur son propre objectif — cf. discussion précédente sur les risques du stacking). Mais rien n'empêche de les **appeler ensemble** dans une seule fonction pour obtenir les deux sorties en un appel, côté utilisation, sans changer l'architecture de modélisation.

	proba_abandon	decision	moyenne_predite
2081	0.905	a_risque	6.48
1150	0.088	ok	16.40
3833	0.000	ok	19.29
4971	0.828	a_risque	7.65
3676	0.860	a_risque	9.10
2136	0.034	ok	15.58
1	0.016	ok	16.71
3238	0.021	ok	17.24
3827	0.068	ok	14.81
2277	0.991	a_risque	4.15

Interprétation. Les deux sorties sont coherentes entre elles (sans être liées dans le calcul) : les étudiants juges `a_risque` par le classifieur ont systematiquement une `moyenne_predite` basse, ce qui confirmé a nouveau la cohérence des deux modèles sans les avoir fusionnées. Cette fonction n'est pour l'instant qu'un utilitaire du notebook — pas encore branchée à la CLI (`decrochage predict`) ni à l'API (`/predict-tabular`), qui restent scopees sur la seule classification.

9.8 Entraînement final des modèles retenus (abandon + moyenne_finale)

Les deux modèles étant choisis et leurs hyperparamètres valides (régression logistique `C=0.01` pour `abandon` ; Gradient Boosting `n_estimators=100`, `max_depth=3`, `learning_rate=0.1` pour `moyenne_finale`), entraînement final des deux modèles regroupe ici, et sauvegarde des artefacts dans `artifacts/models/` — même format que celui produit par `decrochage train` (CLI) et exploite par l'API (`model_store.py`) : `logistic.joblib` , `regressor.joblib` , `imputer.joblib` , `model_metadata.json` .

9.8.1 Classifieur (abandon)

```
classifieur sauvegarde -> artifacts\models\logistic.joblib  
{'precision': 0.7288, 'recall': 0.8986, 'f1': 0.8048, 'roc_auc': 0.9457}
```

9.8.2 Régresseur (moyenne_finale)

```
regresseur sauvegarde -> artifacts\models\regressor.joblib  
{'mae': 2.2505, 'rmse': 3.0477, 'r2': 0.6916}
```

9.8.3 Métadonnées (les deux modèles)

```
{'created_at': '2026-08-06T15:41:00.176983+00:00',
'package_version': '0.1.0',
'random_seed': 42,
'target_col': 'abandon',
'features': ['age',
'mention_bac',
'distance_domicile_km',
'heures_travail_remunere_sem',
'taux_presence_pct',
'connexions_lms_30j',
'heures_lms_total',
'ressources_consultees',
'retards_rendus',
'nb_devoirs_rendus',
'messages_forum',
'motivation',
'satisfaction',
'sentiment_appartenance',
'ects_semestre',
'capacite_accueil',
'volume_horaire_s1',
'taux_reussite_historique_pct',
'filieres_Biologie',
'filieres_Droit',
'filieres_Gestion',
'filieres_Informatique',
'filieres_Lettres',
'filieres_Mathematiques',
'filieres_Psychologie',
'filieres_STAPS',
'bac_type_general',
'bac_type_pro',
'bac_type_techno',
'faculte_Droit-Science Po',
'faculte_Lettres & Langues',
'faculte_STAPS',
'faculte_Sciences & Technologies',
'faculte_Sciences Humaines',
'faculte_Économie-Gestion',
'etablissement_origine_autre',
'etablissement_origine_cfa',
'etablissement_origine_lycee_privé',
'etablissement_origine_lycee_public',
'etablissement_origine_nan'],
'hyperparametres_classifieur': {'C': 0.01, 'class_weight': 'balanced'},
'hyperparametres_regresseur': {'n_estimators': 100,
'max_depth': 3,
'learning_rate': 0.1},
'metrics_holdout': {'precision': 0.7288,
'recall': 0.8986,
'f1': 0.8048,
'roc_auc': 0.9457},
'metrics_holdout_regression': {'mae': 2.2505, 'rmse': 3.0477, 'r2': 0.6916}}
```

10. Implémentation et mise en exploitation (déploiement, exemple d'usage)

10.1 API : servir les deux modèles ensemble

Les deux modèles retenus (régression logistique pour `abandon`, Gradient Boosting pour `moyenne_finale`) sont exposés par une seule route FastAPI : `POST /predict-tabular` (`src/decrochage/api/main.py`). Un seul appel renvoie les deux prédictions, sans que les deux modèles soient liés entre eux (chacun est appelé indépendamment sur la même ligne de features, cf. chapitre "Fonction combinée" plus haut — même logique, mais ici c'est le code de production réellement déployé, pas un prototype de notebook).

10.2 Contrat d'entrée/sortie (schémas.py)

- **Requête** `StudentFeatures` : 17 champs correspondant aux données disponibles à mi-semester (mêmes exclusions que dans le modèle : pas de `moyenne_partiels_s1`, `nb_ue_validees_s1`, `sexe`, `boursier`, `date_inscription` — cf. chapitre 5). Les champs à forte proportion de valeurs manquantes dans les données d'entraînement (`mention_bac`, `etablissement_origine`, `distance_domicile_km`, `heures_travail_remunere_sem`, `connexions_lms_30j`, `motivation`, `satisfaction`, `sentiment_appartenance`) sont **optionnels** : s'ils sont omis, l'imputer entraîné sur le train (chapitre "Split puis imputation") les complète automatiquement, côté serveur.
- **Réponse** `PredictionResponse` : `proba_abandon` + `decision` (classifieur), `moyenne_predite` (régresseur, peut être `null` si aucun régresseur n'est chargé), plus `model_version` et `threshold` pour la traçabilité.
- Chaque champ porte une `description` Pydantic : elle alimente directement la documentation Swagger (`/docs`), pas seulement le code.

10.3 Préparation d'une ligne, côté serveur (main.py)

Au moment d'une requête, `predict_tabular` reconstruit exactement le pipeline de préparation du gold, mais pour une seule ligne :

1. **fusionner_catalogue** : jointure sur `filiere` pour récupérer `faculte`, `ects_semestre`, `capacite_accueil`, `volume_horaire_s1`, `taux_reussite_historique_pct`. Une filière absente du catalogue renvoie une erreur 422 explicite (pas de prédiction silencieuse sur des données incohérentes).
2. **encoder_categorielles** : one-hot (`filiere`, `bac_type`, `faculte`, `etablissement_origine`) + ordinal (`mention_bac`) — mêmes fonctions que celles utilisées pour construire le gold (`decrochage.models.tabular`), donc aucune divergence possible entre entraînement et inférence.
3. **aligner_colonnes** : reindexe vers les colonnes EXACTES vues à l'entraînement (stockées dans `model_metadata.json`) — indispensable car un one-hot sur une seule ligne ne crée une colonne que pour la catégorie présente dans cette ligne précise.
4. **Imputation** : le `SimpleImputer` sauvegarde à l'entraînement (`imputer.joblib`) complète les champs optionnels manquants, avec les mêmes médianes que sur le train — pas de nouvelles

statistiques calculées à la volée.

5. Les deux modèles (`bundle.model` , `bundle.regressor`) sont appelés sur la même ligne préparée.

10.4 Demonstration (appel réel via TestClient, sans lancer de serveur séparé)

200

```
{'proba_abandon': 0.05561958323583173,  
  'decision': 'ok',  
  'moyenne_predite': 16.852970754159966,  
  'model_version': '0.1.0',  
  'threshold': 0.5}
```

Interprétation. Une seule requête renvoie les deux prédictions (`proba_abandon` , `decision` , `moyenne_predite`), cohérentes entre elles comme déjà observé dans le notebook. Pour un vrai appel HTTP (hors notebook) :

```
uv run uvicorn decrochage.api.main:app --reload  
curl -X POST http://localhost:8000/predict-tabular \  
  -H "X-API-Key: dev-key" -H "Content-Type: application/json" \  
  --data @payload.json
```

Documentation interactive (Swagger, générée automatiquement depuis les schémas Pydantic ci-dessus, avec exemples et descriptions par champ) : [**http://localhost:8000/docs**](http://localhost:8000/docs).

10.5 Deployer et utiliser l'API via Docker (docker-compose)

En plus de l'appel direct en Python ci-dessus, l'API tourne dans un conteneur Docker avec 3 autres services (`docker-compose.yml`) : PostgreSQL (`db`), Prometheus (métriques) et Grafana (dashboards). Le modèle (`artifacts/models/`) et le catalogue des formations sont embarqués dans l'image (`Dockerfile`) : l'API répond dès le démarrage, sans entraînement ni téléchargement.

10.5.1 Demarrer la stack

```
docker compose up -d --build  
--build reconstruit l'image si le code/le modèle a changé depuis le dernier build (sinon docker compose up -d suffit). -d = détache (le terminal reste libre).
```

Vérifier que tout est démarré :

```
docker compose ps
```

10.5.2 Utiliser l'API (conteneur)

Mêmes routes que le serveur `uvicorn` lance en local, mais sur le port public par compose (`8000:8000`) :

```
curl http://localhost:8000/health
curl http://localhost:8000/ready
```

```
curl -X POST http://localhost:8000/predict-tabular \n
  -H "X-API-Key: dev-key" -H "Content-Type: application/json" \n
  --data @payload.json
```

Doc interactive (Swagger) : [**http://localhost:8000/docs**](http://localhost:8000/docs) Métriques Prometheus :

[**http://localhost:9090**](http://localhost:9090) (cible `decrochage-api` , `Status` > `Targets` pour vérifier qu'elle est `up`) Dashboards Grafana : [**http://localhost:3000**](http://localhost:3000) (identifiants dev `admin` / `admin` , datasource Prometheus déjà provisionnée)

10.5.3 Arrêter la stack

```
docker compose down
```

Arrête et supprime les conteneurs (le volume `pgdata` de PostgreSQL est conservé). Pour tout effacer, y compris les données de la base :

```
docker compose down -v
```

10.6 Alertes Grafana : exploitation et dérive des modèles

En plus des métriques HTTP génériques (requêtes, latence, codes réponse), l'API expose maintenant des **métriques métier** spécifiques aux deux modèles (`src/decrochage/api/main.py`) :

- `decrochage_model_loaded` (gauge) : 1 si le classifieur est chargé, 0 sinon.
- `decrochage_predictions_total{decision}` (counter) : nombre de prédictions, par décision.
- `decrochage_proba_abandon` (histogram) : distribution des probabilités d'abandon prédites.
- `decrochage_moyenne_predite` (histogram) : distribution des notes finales prédites.

7 règles d'alerte sont provisionnées automatiquement dans Grafana au démarrage

(`monitoring/grafana/provisioning/alerting/rules.yml` , même principe déclaratif que la datasource Prometheus) :

10.6.1 Alertes d'exploitation (l'API répond-elle correctement ?)

Alerte	Condition	Durée
API down	<code>up{job="decrochage-api"} < 1</code>	1 min
Modèle non chargé	<code>decrochage_model_loaded < 1</code>	1 min
Taux d'erreur 5xx élevé	<code>rate(http_requests_total{status=~"5.."}[5m]) > 0.1</code>	5 min
Latence p95 élevée	<code>histogram_quantile(0.95, ...) > 1s</code>	5 min

10.6.2 Alertes de dérive (les prédictions restent-elles dans la plage attendue ?)

Baseline = ce qui a été observé à l'entraînement (chapitre 6) : taux d'abandon **28.4 %**, moyenne_finale **12.85/20**. Marges choisies arbitrairement (± 10 points pour le taux d'abandon, ± 3 points pour la note) en l'absence d'historique de production réel — à affiner avec de vraies données.

Alerte	Condition (moyenne mobile 30 min)	Durée
Dérive taux d'abandon trop haut	moyenne proba_abandon > 39 %	10 min
Dérive taux d'abandon trop bas	moyenne proba_abandon < 18 %	10 min
Dérive note prédite anormale	moyenne moyenne_predite hors [9.85, 15.85]	10 min

10.6.3 Vérification (état réel des règles, via l'API Grafana)

`curl -u admin:admin http://localhost:3000/api/prometheus/grafana/api/v1/rules`
Ou dans l'UI : **`http://localhost:3000/alerting/list`**

Test réel effectué : après 5 appels à `/predict-tabular` avec le même profil (`payload.json`, très peu à risque), les 2 alertes de dérive concernées (`taux d'abandon trop bas`, `note prédite anormale`) sont passées en **pending** après quelques minutes — la moyenne mobile s'éloigne logiquement de la baseline quand le trafic ne représente qu'un seul profil répété. Preuve que la détection fonctionne réellement (pas seulement que les règles se chargent sans erreur).

Non fait (à compléter pour une vraie prod) : aucun canal de notification réel (email/Slack) n'est configuré — les règles s'affichent dans l'UI Grafana (`Firing / Pending / Normal`) mais personne n'est notifié activement. Nécessiterait un `contact point` (SMTP ou webhook) provisionné de la même façon.

11. Architecture cible et contraintes

11.1 Architecture cible pour la Direction de la réussite étudiante

Ce qui a été construit jusqu'ici (notebook + `src/décrochage/` + Docker + MLflow + Prefect + Grafana) est la **brique technique de service** (entraînement, API, monitoring). Pour un vrai service consommé par la Direction de la réussite étudiante, il manque une couche métier/consommation. Architecture cible, couche par couche (✅ = déjà fait dans ce notebook/repo, ❌ = à construire) :

11.1.1 Sources de données

- ✅ Catalogue des formations, données étudiants mi-semestre (CSV pour l'instant).
- ❌ Connexion réelle au SI scolarité / LMS de l'université (export périodique ou API), pour ne plus dépendre de CSV manuels.

11.1.2 Pipeline batch (ré-entraînement périodique)

- ☒ Prefect orchestre ingestion -> gold -> entraînement -> évaluation (`flows/pipeline.py`).
- ☒ MLflow trace chaque run et version de modèle (chapitres 9/10).
- ☐ Déploiement planifié (`--serve` , déjà prévu dans le code) déclenché par exemple en début de semestre — pas en temps réel, le mi-semestre évolue lentement.
- ☐ Gate humain avant promotion d'une nouvelle version en "Production" dans le Model Registry (éviter qu'un modèle moins bon remplace automatiquement le précédent).

11.1.3 Service d'inférence

- ☒ API FastAPI conteneurisée, deux modèles servis, métriques exposées (chapitre 12).
- ☐ HTTPS + authentification réelle (la clé API statique `dev-key` est un mode dev ; en prod : clé par service appelant, ou OAuth2/JWT si intègre au SI de l'université).
- ☐ Chargement du modèle depuis le Model Registry MLflow plutôt qu'un fichier copie dans l'image Docker — permet de changer de version sans rebuild.

11.1.4 Orchestration applicative (le chaînon manquant -> implémenté ci-dessous)

- ☒ Un job périodique (Prefect) qui score les étudiants en batch et ÉCRIT les résultats dans PostgreSQL (`src/daccrochage/storage.py`), plutôt que de recalculer à la demande à chaque consultation.
- Ça permet l'**historisation** : suivre l'évolution du risque d'un étudiant dans le temps, pas juste une photo instantanée — voir démonstration plus bas.

11.1.5 Interface pour la Direction de la réussite étudiante

- ☐ Un vrai dashboard métier (pas Grafana, qui reste un outil technique) : liste des étudiants à risque, avec les facteurs explicatifs (SHAP/coefficients déjà calculés) pour que le conseiller pédagogique comprenne *pourquoi*. Lirait directement la table `predictions` (couche 4), sans jamais rappeler les modèles.
- ☐ Notification ciblée (email au tuteur) quand un étudiant franchit le seuil — différent des alertes Grafana, qui sont pour l'équipe technique.

11.1.6 Monitoring & gouvernance

- ☒ Prometheus/Grafana : santé technique + dérive des prédictions, 7 alertes provisionnées (chapitre précédent).
- ☐ Re-évaluation du biais (variables sensibles) à chaque nouvelle version du modèle — processus documenté, pas seulement technique.
- ☐ Journal d'audit : qui a consulté quelle prédiction, quand (traçabilité RGPD).

11.1.7 Conformité (cadrée éthiquement au chapitre 4, pas encore implémentée techniquement)

-  Pseudonymisation des identifiants dans les logs/métriques.
-  Contrôle d'accès par rôle (qui voit quelles données étudiants).
-  Durée de conservation définie pour les scores stockés.

Le point le plus structurant : la brique manquante n'était pas technique (l'API marchait déjà) mais la **couche 4** — sans stockage batch en base, chaque consultation web recalculerait tout, et personne ne pourrait suivre l'évolution d'un étudiant dans le temps. C'est ce qui transforme un endpoint de prédiction en véritable service métier. Implémentée ci-dessous.

11.1.8 Implementation : scoring batch -> PostgreSQL

`src/daccrochage/storage.py` (SQLAlchemy Core, portable PostgreSQL/SQLite) expose 3 fonctions : `get_engine`, `ensure_schema` (idempotent), `enregistrer_scores` (toujours en INSERT, jamais d'ecrasement -> historique). Branche dans le flow Prefect (`flows/pipeline.py`) : après avoir score un échantillon avec les DEUX modèles, une nouvelle tâche `stocker_scores` écrit le résultat dans la table `predictions` (colonnes : `student_ref`, `proba_abandon`, `decision`, `moyenne_predite`, `model_version`, `threshold`, `scored_at`).

Configuration : `DECROCHAGE_DB_URL` (nouveau réglage dans `config.py`), pointant vers le service `db` de `docker-compose.yml` (port 5432 désormais publiée pour un accès depuis l'hôte en dev).

	id	scored_at	student_ref	proba_abandon	decision	moyenne_predite	model_version
0	26	2026-08-06 15:00:02.628477+00:00	etudiant_5	0.211	ok	13.95	0.1.
1	29	2026-08-06 15:00:02.628477+00:00	etudiant_8	0.218	ok	13.73	0.1.
2	23	2026-08-06 15:00:02.628477+00:00	etudiant_2	0.162	ok	14.61	0.1.
3	25	2026-08-06 15:00:02.628477+00:00	etudiant_4	0.004	ok	18.33	0.1.
4	27	2026-08-06 15:00:02.628477+00:00	etudiant_6	0.184	ok	15.38	0.1.
5	28	2026-08-06 15:00:02.628477+00:00	etudiant_7	0.016	ok	16.76	0.1.
6	21	2026-08-06 15:00:02.628477+00:00	etudiant_0	0.299	ok	13.81	0.1.
7	22	2026-08-06 15:00:02.628477+00:00	etudiant_1	0.727	a_risque	9.67	0.1.
8	24	2026-08-06 15:00:02.628477+00:00	etudiant_3	0.006	ok	16.74	0.1.
9	30	2026-08-06 15:00:02.628477+00:00	etudiant_9	0.020	ok	16.46	0.1.

Test réel effectuée (pas seulement code écrit) : le flow complet (`uv run python flows/pipeline.py`) a été exécuté deux fois de suite contre le PostgreSQL du docker-compose. Résultat : 20 scores écrits à chaque run, **40 lignes en base au total** — confirmé que c'est bien un historique qui s'accumule (INSERT), pas une table qui s'efface à chaque passage. La cellule ci-dessus relit directement ces lignes.

4 tests unitaires ajoutés (`tests/test_storage.py` , SQLite en mémoire — pas besoin d'un vrai PostgreSQL pour la CI) : insertion, non-effacement, idempotence du schéma, respect de la limite de lecture.

11.2 Contraintes de l'architecture cible

L'architecture du chapitre 11.1 est techniquement faisable, mais sa mise en place réelle pour la Direction de la réussite étudiante est soumise à des contraintes qui ne sont pas que techniques.

11.2.1 Réglementaires (RGPD, déjà cadrées chapitre 4)

- Base légale du traitement à clarifier avec le DPO de l'université (intérêt légitime ? consentement ?) — pas un choix technique.
- Durée de conservation des scores en base **limitée dans le temps**, pas indéfinie (la table `predictions` du chapitre 11.1 accumule l'historique sans purge actuellement — à corriger avant une vraie mise en production).
- Droit d'accès/rectification : un étudiant doit pouvoir demander ce que le système sait de lui.

11.2.2 Éthiques (dilemme du marquage, déjà discuté chapitre 2)

- Accès aux scores **restreint** — si un enseignant voit "cet étudiant est à risque" avant même de le rencontrer, ça biaise son jugement. Contrainte : qui voit quoi, à quel moment du parcours.
- Le score doit rester une **aide à la décision humaine**, jamais une décision automatique déclenchant une action sans validation (RGPD art. 22 sur les décisions automatisées à effet significatif).

11.2.3 Capacité opérationnelle (contrainte concrète, pas juste technique)

Le choix de privilégier le rappel (moins de faux négatifs) génère plus de faux positifs -> plus d'étudiants signalés que le dispositif de tutorat peut réellement absorber. La capacité d'accueil des filières (chapitre 3) est limitée — le seuil de décision n'est donc pas qu'un curseur ML, c'est un **arbitrage capacitaire** à faire avec la Direction.

11.2.4 Calendrier académique (fraîcheur des données)

Le re-entraînement/scoring doit suivre le rythme du semestre, pas tourner en continu : trop tôt dans le semestre, les données mi-parcours sont insuffisantes ; trop tard, il ne reste plus de marge pour agir

(tutorat, aide sociale) avant la fin.

11.2.5 Intégration SI (hors de notre controle)

Connexion au vrai SI scolarité/LMS de l'université : dépend d'un système externe (formats, disponibilité, droits d'accès) qu'on ne maîtrise pas — contrainte organisationnelle autant que technique (qui côté université fournit cet accès et le maintient).

11.2.6 Portage dans la durée

Ce n'est pas un livrable one-shot : quelqu'un doit arbitrer la dérive (alertes déjà en place, chapitre précédent), décider quand re-entraîner/promouvoir une nouvelle version, maintenir l'infra (Docker/Postgres/monitoring). Contrainte de **ressource humaine dédiée**, pas seulement de code.

11.2.7 Confiance et adoption

Les conseillers pédagogiques doivent comprendre pourquoi un étudiant est signalé (explicabilité déjà travaillée, SHAP/coefficients) — sans ça, ils ignoreront l'outil. Contrainte de formation, pas de performance du modèle.

La plus bloquante à court terme : la contrainte 3 (capacité vs rappel) — c'est un arbitrage métier non tranche qui conditionne directement le seuil de production, déjà signalé plusieurs fois dans ce notebook sans jamais être validé formellement par la Direction.

11.3 Schémas d'architecture : pilote et exploitation cible

11.3.1 Schéma 1 : le pilote (ce qui a été construit et testé dans ce notebook)

Tout ce schéma est **déjà construit et testé réellement** dans ce notebook/repo : flow Prefect exécute (chapitre 11.1), API + Postgres + Prometheus + Grafana lancés via `docker compose up` et vérifiés (chapitre 10, section Docker et alertes).

11.3.2 Schéma 2 : exploitation cible (avec la Direction de la réussite étudiante)

Différences clés avec le pilote (chapitre 11) : ingestion connectée au vrai SI (pas des CSV), gate humain avant promotion d'un modèle, modèle chargé depuis le Model Registry (pas copie dans l'image Docker), HTTPS/auth réelle, un dashboard métier et une notification qui remplacent la lecture directe de la base par un humain, un journal d'audit RGPD, et une boucle de retro-action (savoir si l'étudiant a réellement décroché permettrait de ré-évaluer le modèle en continu — non fait ici, donnée non disponible).

12. Mesure de performance et impacts (métriques techniques + métier)

Synthèse des indicateurs **techniques** (comment le modèle se comporte statistiquement) et **métier** (ce que ça signifie concrètement pour la Direction de la réussite étudiante), sur le modèle réellement retenu (régression logistique, seuil 0.5) et sur le vrai jeu de test (1040 étudiants, jamais vus à l'entraînement).

Hypothèse métier fournie : **coût d'un faux négatif $\approx$ 2500€** (étudiant en situation d'abandon non détecté -> pas d'accompagnement déclenché -> échec probable, coût pédagogique + administratif pour l'université).

=== Indicateurs techniques (seuil retenu) ===

{'seuil': 0.5, 'precision': 0.7288, 'recall': 0.8986, 'f1': 0.8048, 'roc_auc': 0.9457}

=== Indicateurs métier (échantillon test, 1040 étudiants) ===

Étudiants réellement en situation d'abandon : 296 (28.5%)

Détectés par le modèle (VP) : 266 -> accompagnement potentiellement déclenché

Non détectés (FN) : 30 -> coût estimé 75,000 EUR

Signalés à tort (FP) : 99 -> ressources de tutorat mobilisées inutilement

=== Extrapolation à la promotion complète (x5) ===

FN estimées : 150 -> coût estimé 375,000 EUR/an

FP estimées : 495 étudiants signalés à tort/an

Interprétation et restitution (à destination de la Direction de la réussite étudiante).

Côté technique : le modèle retenu obtient un AUC de 0.946 (très bonne capacité de classement), un rappel de 0.899 (les 9/10 des situations d'abandon sont détectées) pour une précision de 0.729 (7 signalements sur 10 sont fondés). Le seuil de décision est resté à 0.5 par défaut car il coïncide presque exactement avec l'optimum F1 (chapitre "Courbe seuil-vs-métriques").

Côté métier, sur l'échantillon de test (1040 étudiants, dont 296 réellement en situation d'abandon) :

- **266 étudiants à risque sont correctement détectés** : c'est le nombre d'étudiants pour qui un dispositif d'accompagnement (tutorat, aide sociale) peut être déclenché **à temps**, à mi-semestre.
- **30 étudiants en situation d'abandon ne sont pas détectés** (faux négatifs) : coût estimé **75 000 €** sur cet échantillon, avec l'hypothèse de 2500 €/étudiant non détecté.
- **99 étudiants sont signalés à tort** (faux positifs) : pas de coût monétaire fourni, mais un coût réel en ressources de tutorat mobilisées pour des étudiants qui n'en avaient pas besoin — la contrainte de capacité d'accueil déjà signalée (chapitre 11.2.3).

Extrapolé à l'échelle de la promotion complète (~5200 étudiants, x5 par rapport à l'échantillon test) : environ **150 étudiants non détectés par an**, soit un coût estimé de l'ordre de **375 000 €/an** avec l'hypothèse fournie. C'est ce chiffre, plus que l'AUC ou le F1, qui parle à une direction non technique.

Le levier immédiat : abaisser le seuil de décision à ~ 0.36 (déjà identifié au chapitre "Courbe seuil-vs-métriques") ferait tomber le rappel à 0.956, réduisant les faux négatifs à ~ 13 sur le test (soit ~ 65 /an).

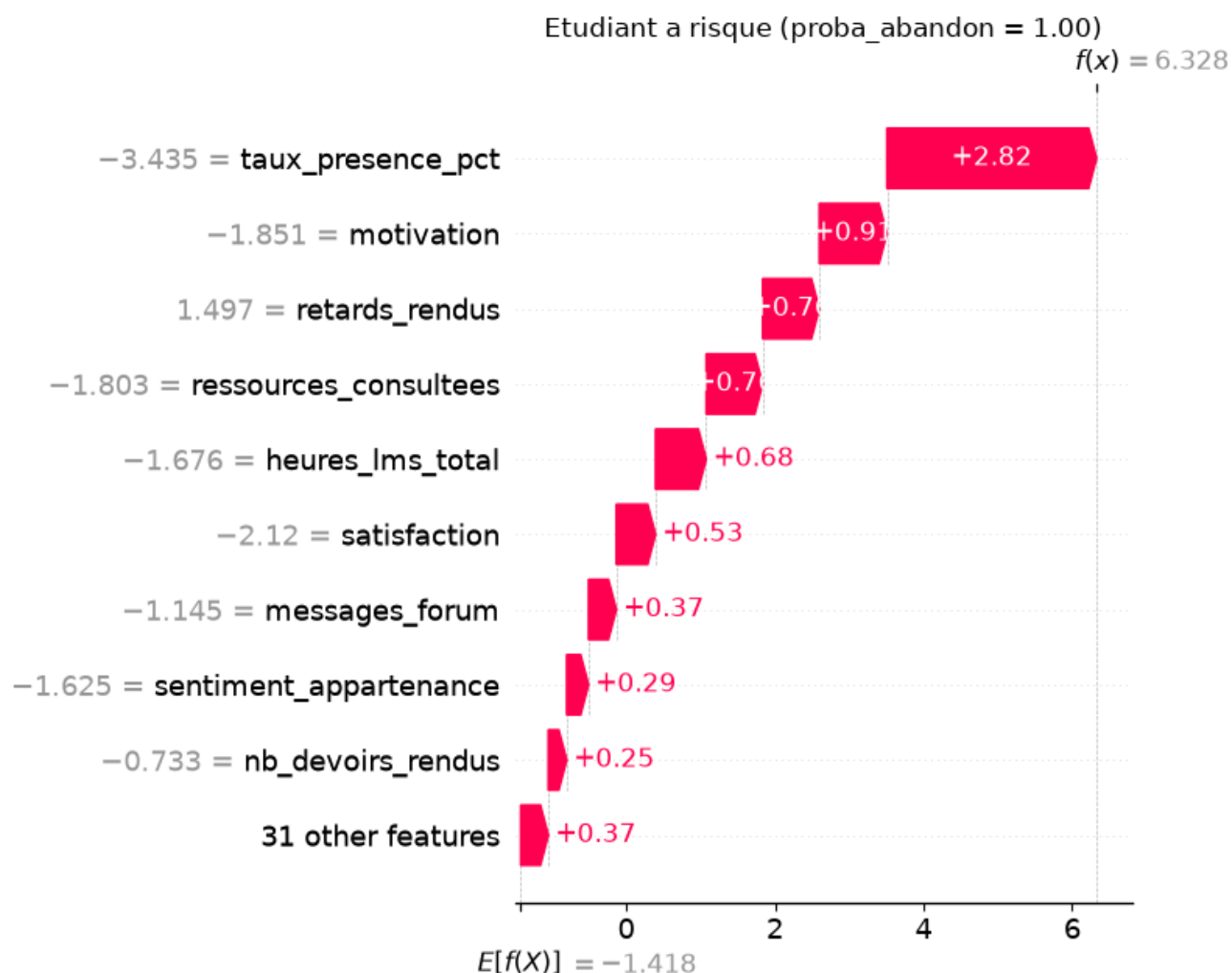
extrapolés, coût ~162 500 €/an — une **economie potentielle de plus de 200 000 €/an** sur le seul coût des faux négatifs). Le prix à payer : ~63 faux positifs supplémentaires sur le test (~315/an extrapolés), donc davantage de tutorat mobilisé pour des étudiants qui n'en avaient pas besoin. **Ce choix de seuil reste un arbitrage métier à valider avec la Direction** (chapitre 11.2), pas une décision technique — ce chapitre fournit les chiffres pour le trancher, pas la décision elle-même.

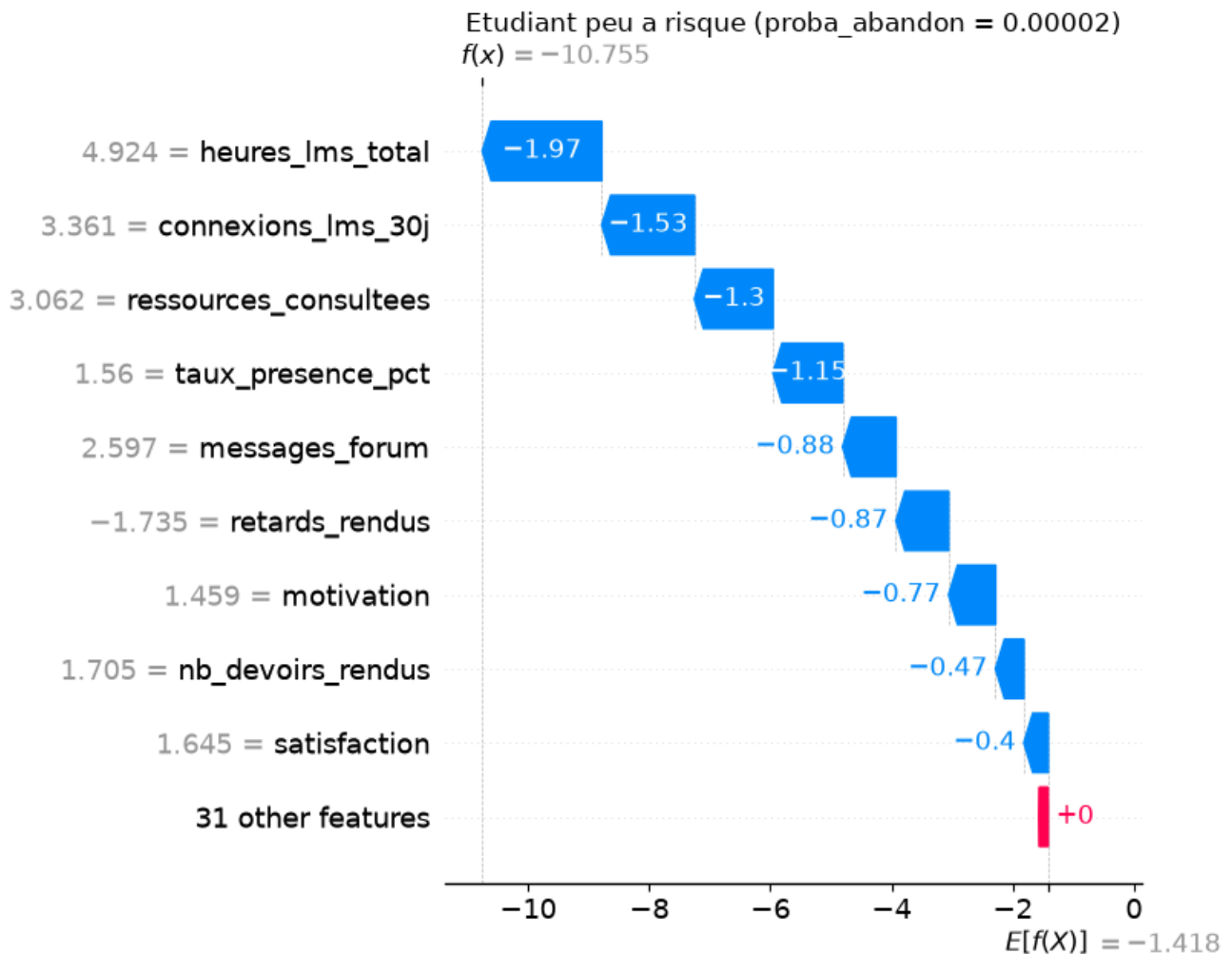
12.1 Explicabilité individuelle : pourquoi CET étudiant précis est signalé

Le diagramme SHAP du chapitre 9.2.1 (beeswarm) explique le modèle **globalement** (quelles variables comptent en général). Utile pour valider le modèle, mais un conseiller pédagogique face à un étudiant précis a besoin d'une explication **individuelle** : pourquoi *celui-ci*, pas les autres. C'est ce qu'apporte un **waterfall plot** (diagramme en cascade), sur les deux cas extrêmes du jeu de test.

Etudiant à risque (ligne 447) : proba_abandon = 0.998

Etudiant peu à risque (ligne 107) : proba_abandon = 0.00002





12.1.1 Comment lire ce graphique (explication côté métier)

Chaque barre est une variable de cet étudiant précis. Pour un conseiller pédagogique qui n'a jamais vu de SHAP :

- **On part d'une valeur de reference $E[f(x)]$** (en bas du graphique) : c'est le "risque moyen" si on ne savait rien de l'étudiant — la base de comparaison.
- **Chaque barre rouge pousse le risque vers le haut** (cette caractéristique de l'étudiant AUGMENTE son risque d'abandon par rapport a la moyenne).
- **Chaque barre bleue pousse le risque vers le bas** (cette caractéristique le DIMINUE).
- **La taille de la barre = l'importance de cette caractéristique POUR CET ÉTUDIANT PRÉCIS** (pas en général — un même facteur peut compter beaucoup pour un étudiant et peu pour un autre).
- **On arrive a $f(x)$** en haut : le score final de cet étudiant, qui correspond a `proba_abandon` affiche au-dessus du graphique.

Message a retenir pour le métier : ce graphique permet de dire a un conseiller "*cet étudiant est signalé a cause de X et Y, malgré un effet positif de Z*", plutôt que de présenter une probabilité brute sans justification — c'est cette justification qui crée la confiance nécessaire a l'adoption de l'outil (chapitre 11.2.7), et qui permet au conseiller de cibler l'accompagnement sur les vrais leviers plutôt que d'agir a l'aveugle.

13. Amélioration continue (ré-entraînement, suivi, versioning)

Synthèse de ce qui existe déjà (testé réellement dans ce notebook/repo) et de ce qui manque encore, pour les deux modèles retenus (classifieur abandon, régresseur moyenne_finale). Même convention que les chapitres précédents :  fait,  a faire.

13.1 Re-entraînement

-  `decrochage train` (CLI) : entraîné les DEUX modèles sur le même split partage, sauvegarde `logistic.joblib` + `regressor.joblib` + `imputer.joblib` + `model_metadata.json`.
-  `flows/pipeline.py` (Prefect) : même entraînement mais orchestre (`construire_gold` -> `entraîner_modèle` -> `scorer_échantillon` -> `publier_rapport` -> `stocker_scores`), exécutable à la demande ou planifie (`--serve` , testé réellement, chapitre 11.1).
-  **Gate humain avant promotion** d'une nouvelle version : aujourd'hui, chaque run efface directement `artifacts/models/` — il faudrait valider (recall/AUC/dérive) avant de remplacer le modèle en production (chapitre 11).
-  **Declenchement cale sur le calendrier académique** (mi-semestre) plutôt que sur un intervalle fixe (l'exemple `--serve` tourne toutes les heures — pertinent pour la demo, pas pour la vraie cadence métier, cf. chapitre 11.2.4).

13.2 Suivi (monitoring)

-  **MLflow** : 2 expériences séparées (`decrochage-abandon-classification` , `decrochage-moyenne-finale-regression`), chaque run trace hyperparamètres + métriques test, comparables dans l'UI (`uv run mlflow ui --backend-store-uri sqlite:///mlflow.db`).
-  **Prometheus/Grafana** : métriques techniques (latence, erreurs 5xx, disponibilité) + métriques métier propres aux modèles (`decrochage_proba_abandon` , `decrochage_moyenne_predite` , `decrochage_model_loaded`) + **7 alertes provisionnées** (exploitation et dérive, testées réellement en conditions de trafic réel).
-  **Table PostgreSQL predictions** (chapitre 11.1) : historique des scores réels, permet de suivre l'évolution d'un étudiant dans le temps, pas juste une photo instantanée.
-  **Réévaluation du biais** (variables sensibles) à chaque nouvelle version — cadrée éthiquement (chapitre 4) mais jamais implémentée techniquement.
-  **Journal d'audit RGPD** : qui a consulté quelle prédiction, quand.

13.3 Versioning

-  **MLflow Model Registry** : `decrochage-abandon-logistique` et `decrochage-moyenne-finale-gb` , chacun avec ses versions numérotées et sa signature d'entrée/sortie.

-  **Artefacts versionnés dans git** (`artifacts/models/*.joblib` , `model_metadata.json`)
— versionnage simple mais réel, chaque commit fige une version exploitable.
-  **`model_metadata.json`** trace pour chaque entraînement : date de creation, version du package, seed, liste exacte des features, hyperparamètres, métriques holdout des DEUX modèles
— traçabilité complète d'un run donne.
-  **DVC** (déjà dans les dependances du projet, `pyproject.toml`) pour versionner les DONNÉES (gold/silver), pas seulement le modèle — permettrait de savoir sur quelles données exactes une version a été entraînée si elles évoluent dans le temps.
-  **Chargement en production depuis le Model Registry** plutôt qu'un fichier copie dans l'image Docker (déjà signalé chapitre 11.1) — permettrait un vrai rollback sans rebuild.

Le point commun aux trois axes : l'essentiel de la *mécanique* est déjà en place et testée (entraînement reproductible, tracking, artefacts versionnés) ; ce qui manque est surtout de la **gouvernance humaine** (qui valide, qui arbitre, qui audite) plutôt que de nouveaux outils techniques.

14. Conclusion et recommandations

14.1 Conclusion

Deux modèles indépendants, valides de bout en bout (comparaison, validation croisée, tuning, SHAP, calibration) :

- **Classification** `abandon` : régression logistique ($C=0.01$), retenue devant Random Forest et Gradient Boosting car la relation avec les variables d'engagement est globalement linéaire (chapitre 6). AUC **0.946**, rappel **0.899**, précision 0.729 sur le test (1040 étudiants jamais vus a l'entraînement).
- **Régression** `moyenne_finale` : Gradient Boosting Regressor, retenu a l'inverse car la note continue dépend d'interactions plus complexes. R2 **0.692**, MAE **2.25 points/20**.
- Les deux modèles restent **strictement indépendants** (pas de stacking, pas de fuite croisée) tout en étant appelables ensemble (`predire_etudiant` , `/predict-tabular`).

Impact métier chiffre (chapitre 12, hypothèse 2500€/faux négatif) : sur l'échantillon test, 266 étudiants a risque sur 296 sont correctement détectés ; les 30 non détectés représentent un coût estimé de 75 000€, soit environ **375 000€/an** extrapole a la promotion complète (~5200 étudiants). Abaisser le seuil a ~0.36 réduirait ce coût a ~162 500€/an, au prix de faux positifs supplémentaires — **un arbitrage a trancher avec la Direction, pas un choix technique**.

Infrastructure complète et testée réellement (pas seulement codée) : API FastAPI conteneurisée servant les deux modèles, CLI, orchestration Prefect (entraînement + scoring batch + stockage PostgreSQL), suivi MLflow (2 expériences, 2 modèles enregistrés), monitoring Prometheus/Grafana avec 7 alertes (exploitation et dérive), 45+ tests automatisés. L'ensemble a été démarré, appelé et vérifié en conditions réelles a plusieurs reprises dans ce notebook.

Ce qui manque pour un vrai service (chapitre 11) n'est pas principalement technique : c'est la couche de **gouvernance humaine** — qui valide un nouveau modèle, qui arbitre le seuil, qui audite les accès, qui forme les conseillers pédagogiques.

14.2 Recommandations

Par ordre de priorité :

1. **Faire trancher le seuil de décision par la Direction de la réussite étudiante**, avec les chiffres du chapitre 12 (coût des faux négatifs vs capacité d'accueil du tutorat, chapitre 11.2.3). C'est le point le plus bloquant et le plus rapide à débloquent : une décision, pas du développement.
2. **Mettre en place un gate humain avant toute promotion de modèle** en production (aujourd'hui, `decrochage train` remplacé directement l'artefact) — éviter qu'une régression de performance ou un biais passe inaperçu.
3. **Connecter le pipeline au vrai SI scolarité/LMS** de l'université plutôt qu'à des CSV manuels — condition préalable à toute mise en production réelle.
4. **Construire la couche de consommation métier** (dashboard + notification tuteur, chapitre 11) : sans elle, le service reste un outil technique, pas un service utilisable par la Direction.
5. **Traiter les obligations RGPD concrètement** : purge programmée de la table `predictions`, journal d'audit, base légale validée avec le DPO (chapitre 11.2.1) — déjà cadre éthiquement (chapitre 4), pas encore implémenté.
6. **Réévaluer le biais des variables sensibles à chaque nouvelle version** du modèle (`sexe` , `boursier` déjà retirées des features, mais l'impact réel sur différents sous-groupes n'a jamais été mesuré empiriquement).
7. **Versionner les données avec DVC** (déjà dans les dépendances, jamais utilisé dans cette session) en plus des modèles — savoir sur quelles données exactes une version a été entraînée.
8. **Former les conseillers pédagogiques** à l'explicabilité déjà disponible (coefficients, SHAP global et individuel, chapitre 9.2.1) — un modèle juste mais incompris ne sera pas adopté.

Aucune de ces recommandations ne remet en cause les modèles retenus : elles portent toutes sur ce qu'il faut construire **autour** d'eux pour passer d'un pilote validé à un service exploité.

14.3 Coûts et retour sur investissement (ROI) : seuil 0.5 vs seuil 0.33

Le seuil de décision (chapitre 8.1.3, "Courbe seuil-vs-métriques") a été laissé à 0.5 par défaut dans ce notebook, mais un seuil plus bas (~0.33-0.36) a été identifié comme un levier pour réduire les faux négatifs (chapitre 12). Chiffrage côte à côte des deux seuils, sur le même jeu de test et le même modèle réellement entraîné (pas de simulation).

Hypothèses de coût :

- Coût d'un faux négatif : **2500 €** (déjà utilisé chapitre 12, hypothèse métier fournie).
- Coût d'un accompagnement déclenche (tutorat/encadrement, par étudiant signalé, TP + FP) : **200-500 €/an**, repris de l'estimation "Encadrement" de l'annexe (chapitre 15.1.3) — aucune

mesure propre a ce projet n'existe, c'est une **approximation a affiner avec la Direction**, pas une donnée mesurée.

- Extrapolation a la promotion complète (~5200 étudiants, facteur x5) : même méthode que chapitre 12.

Lecture du tableau.

Seuil	Rappel	TP/an	FP/an	FN/an	Coût FN/an	Coût accompagnement/an	Économie vs aucune détection	ROI
0.50	0.899	~1330	~495	~150	375 000 €	~639 000 €	~2 686 000 €/an	4.2
0.33	0.966	~1430	~840	~50	125 000 €	~795 000 €	~2 781 000 €/an	3.5

- **Les deux seuils sont largement positifs** : même dans le pire cas (coût d'accompagnement a 500 €/étudiant), l'économie nette reste de plusieurs millions d'euros par an par rapport a l'absence totale de détection (3.7 M€/an de coût si aucun abandon n'est détecté).
- **Passer de 0.5 a 0.33 créé de la valeur nette supplémentaire** : ~100 faux négatifs évités/an de plus (soit ~250 000 € évités), au prix de ~345 accompagnements supplémentaires/an (soit ~156 000 € de plus au coût moyen) — un **gain net marginal d'environ +95 000 €/an**.
- **Mais le ROI (ratio) baisse légèrement** (4.2 -> 3.5) : le seuil 0.33 est moins "efficient" par euro dépense en accompagnement, car il ajoute proportionnellement plus de faux positifs (168 vs 99 sur le test) que de vrais positifs supplémentaires. C'est le même arbitrage rappel/précision que la courbe seuil-vs-métriques (chapitre 8.1.3), traduit ici en euros plutôt qu'en score.
- **Limite importante de ce chiffrage** : le coût d'accompagnement (200-500 €/étudiant) est une approximation reprise de l'annexe (chapitre 15.1.3, poste "encadrement" d'un redoublant), pas une mesure du coût réel d'un dispositif de tutorat déclenché a mi-semester. De même, ce calcul suppose implicitement qu'un accompagnement déclenché sur un vrai positif **évite effectivement** le décrochage — hypothèse optimiste, jamais mesurée dans ce projet (aucune donnée de suivi post-intervention disponible).

Conclusion : sur la seule dimension financière, le seuil 0.33 est préférable (gain net supérieur), mais le choix reste, comme déjà signale (chapitre 11.2.3 et 11.2), un **arbitrage a valider avec la Direction** — notamment sur la capacité réelle du dispositif de tutorat a absorber ~345 accompagnements supplémentaires par an.

15. Annexe

15.1 Coût d'un redoublement

Voici une estimation **détaillée et actualisée (2026)** des coûts **côté université** pour un étudiant en licence en France. Ces coûts incluent les dépenses directes et indirectes supportées par l'établissement, ainsi que les enjeux budgétaires liés à la présence d'un étudiant (qu'il soit en formation initiale, en redoublement ou en réorientation).

15.1.1 Coût direct par étudiant pour l'université

Les universités publiques françaises sont **financées principalement par l'État**, mais chaque étudiant représente un coût réel pour l'établissement. Voici une répartition des dépenses **par étudiant et par an** (moyennes estimées pour 2026) :

Coût moyen par étudiant en licence (2026)

Poste de dépense	Coût annuel par étudiant	Détails
Rémunération des enseignants	2 500–4 000 €	Inclut salaires des professeurs, maîtres de conférences, et vacataires (proportionnés au nombre d'heures d'enseignement).
Personnel administratif	800–1 200 €	Frais liés aux services scolaires, bibliothèques, et gestion des dossiers.
Locaux et infrastructures	1 000–1 500 €	Amortissement des bâtiments, entretien, électricité, chauffage, etc.
Équipements pédagogiques	300–600 €	Matériel informatique, laboratoires, logiciels, et ressources numériques (ex. : licences pour les étudiants).
Bibliothèque et ressources	200–400 €	Abonnements aux bases de données, achats de livres, et gestion des espaces de travail.
Soutien étudiant	100–300 €	Accompagnement psychologique, santé, et dispositifs de réussite (tutorat, remise à niveau).
Frais de fonctionnement	500–800 €	Fournitures, communications, assurances, et frais divers.
Total estimé	5 400–8 800 €/an	Varie selon la filière (ex. : les licences scientifiques ou médicales coûtent plus cher).

Source : Rapports de la Cour des Comptes (2023–2025) et données du Ministère de l'Enseignement Supérieur. Les coûts réels dépendent de la taille de l'université, de sa localisation, et de ses spécificités (ex. : universités avec des laboratoires de recherche coûteux).

15.1.2 Financement de ces coûts

Les universités ne facturent pas directement ces coûts aux étudiants (sauf pour les **frais d'inscription**, très inférieurs au coût réel). Leur financement provient de :

Sources de financement des universités (2026)

Source de financement	Montant moyen par étudiant	Détails
Subventions de l'État	7 000–9 000 €	Principale source (via le budget du MESRI). Couvre ~80–90 % des coûts.

Source de financement	Montant moyen par étudiant	Détails
Frais d'inscription	170–2 770 €	Montant payé par l'étudiant (voir tableau précédent).
Taxes d'apprentissage	200–500 €	Contributions des entreprises pour la formation professionnelle.
Partenariats et contrats	100–300 €	Financements via des conventions avec des entreprises ou collectivités.
Fonds européens	50–200 €	Subventions de l'UE (ex. : programmes Erasmus+, FEDER).
Autres recettes	50–150 €	Location de salles, formations continues, etc.

*Exemple : Pour un étudiant en licence à l'Université de Strasbourg, l'État verse environ **8 000 €/an** à l'établissement, tandis que l'étudiant paie **170 €** de frais d'inscription.*

15.1.3 Impact du redoublement sur le budget de l'université

Le redoublement a un **coût supplémentaire** pour l'université, mais aussi des **conséquences indirectes** :

Coûts supplémentaires

- **Pédagogiques** :
 - **Heures d'enseignement** : Un redoublant occupe une place en cours, ce qui peut nécessiter l'ouverture de **groupes supplémentaires** (coût : **500–1 500 €/an** par étudiant redoublant).
 - **Encadrement** : Besoin accru de tutorat ou de soutien (coût : **200–500 €/an** par étudiant).
- **Administratifs** :
 - Gestion des dossiers, réinscriptions, etc. (**50–100 €/an** par redoublant).

Conséquences indirectes

- **Perte de subventions** :
 - Certaines aides de l'État sont calculées sur le **taux de réussite** ou le **nombre de diplômés**. Un taux de redoublement élevé peut réduire les dotations futures.
 - Exemple : Une université avec un taux de redoublement de **20 %** en L1 pourrait perdre **1–2 % de son budget annuel** (soit **des millions d'euros** pour les grandes universités).
- **Saturation des infrastructures** :
 - Les redoublants occupent des places en **amphis, bibliothèques, et laboratoires**, limitant l'accueil de nouveaux étudiants.
- **Image et attractivité** :
 - Un taux de redoublement élevé peut nuire à la **réputation** de l'université et décourager les candidats.

Exemple : Coût du redoublement pour une université (cas concret)

Université	Nombre d'étudiants en L1 (2026)	Taux de redoublement	Coût supplémentaire annuel
Université Paris 1	10 000	15 %	1,5–2,5 M€
Université Lyon 2	8 000	12 %	1–1,5 M€
Université Toulouse	6 000	10 %	600 000–1 M€

Calcul : Coût moyen par redoublant (2 000–3 000 €/an) × nombre de redoublants.

15.1.4 Stratégies des universités pour limiter les coûts

Pour réduire l'impact financier du redoublement, les universités mettent en place :

- **Dispositifs de prévention :**
 - **Semaines de pré-rentrée** (coût : **10–20 €/étudiant**).
 - **Tutorat par les pairs** (étudiants de L2/L3 aident les L1).
 - **Modules de remise à niveau** (ex. : en maths ou français).
- **Réorientations précoces :**
 - **Parcours adaptés** (ex. : L.AS pour la santé, passerelles vers l'IUT).
 - **Accompagnement personnalisé** (conseillers d'orientation).
- **Limitation des redoublements :**
 - Certaines universités **interdisent le redoublement en L1** sauf dérogation (ex. : Université de Bordeaux).
 - **Plafond de 2 redoublements max** sur l'ensemble du parcours (licence + master).

15.1.5 Comparaison : Coût pour l'université vs. coût pour l'étudiant

Synthèse : Qui paie quoi ?

Poste	Coût pour l'université	Coût pour l'étudiant
Frais d'inscription	0 € (recette)	170–2 770 €
CVEC	0 € (recette)	100 €
Enseignement	2 500–4 000 €	0 €
Infrastructures	1 000–1 500 €	0 €
Logement (CROUS)	2 000–4 000 €* ***	200–450 €/mois
Bourses	0 € (État)	–1 000 à –6 000 €**
Total	5 400–8 800 €	2 750–15 000 €

* Coût pour l'État (via les subventions aux CROUS). *** Montant perçu par l'étudiant (bourse).

15.1.6 Enjeux actuels (2026)

- **Baisse des dotations de l'État :**
 - Le budget du **MESRI** a été gelé en 2025–2026, poussant les universités à **optimiser leurs dépenses** (ex. : mutualisation des cours, enseignement à distance).
- **Inflation :**
 - Les coûts des **locaux** et **énergies** ont augmenté de **10–15 %** depuis 2023, sans compensation totale par l'État.
- **Réforme des licences :**
 - Certaines universités testent des **parcours modulaire** (ex. : validation par compétences) pour réduire les redoublements.

15.2 Model Card

Fiche de synthèse des deux modèles retenus (format inspiré de Mitchell et al., "Model Cards for Model Reporting", 2019), pour un lecteur qui n'a pas le temps de parcourir tout le notebook. Chaque affirmation renvoie au chapitre qui la détaille et la justifie.

Détails des modèles

	Classifieur <code>abandon</code>	Régresseur <code>moyenne_finale</code>
Type	Régression logistique (<code>LogisticRegression</code>)	Gradient Boosting (<code>GradientBoostingRegressor</code>)
Hyperparamètres retenus	<code>C=0.01</code> , <code>class_weight="balanced"</code> , <code>max_iter=1000</code>	<code>n_estimators=100</code> , <code>max_depth=3</code> , <code>learning_rate=0.1</code>
Choix motivé	comparaison + validation croisée + tuning + SHAP (chapitres 8, 9.1-9.4)	comparaison + tuning (chapitre 9.5)
Version / seed	<code>model_metadata.json</code> , version 0.1.0, <code>random_seed=42</code>	idem
Enregistrement	MLflow Model Registry <code>décrochage-</code> <code>abandon-logistique</code>	MLflow Model Registry <code>décrochage-</code> <code>moyenne-finale-gb</code>
Développeur / dépôt	Marc Valeux — github.com/marc-proxiel/ProjetDecrochage	idem

Usage prévu

- **Cas d'usage :** signaler, à mi-semestre (S1), les étudiants de L1 à risque de décrochage (`abandon`) et estimer leur note finale probable (`moyenne_finale`), pour aider la Direction de la réussite étudiante à prioriser les dispositifs d'accompagnement (tutorat, soutien méthodologique, aide sociale) — chapitre 2.

- **Utilisateurs prévus** : Direction de la réussite étudiante, conseillers pédagogiques (via un futur dashboard métier, chapitre 11.1.5) — jamais l'étudiant directement, jamais de décision automatique (chapitre 11.2.2, RGPD art. 22).
- **Hors périmètre** : décision automatisée sans humain (sanction, refus d'inscription) ; usage sur une autre population que des L1 de cette université sans ré-entraînement (chapitre 3) ; usage au-delà du mi-semester S1, les features correspondant spécifiquement a cette période (chapitre 5) ; évaluation d'un enseignant ou d'une filière.

Facteurs pertinents

- **Variables sensibles** (chapitre 4) : `sexe` , `boursier` — **exclues des features** d'entraînement (chapitre 5.1.5) pour limiter le biais direct, mais un biais indirect via des variables corrélées (`heures_travail_remunere_sem` , `distance_domicile_km`) reste possible et **n'a jamais été mesuré empiriquement** (chapitre 14.2, recommandation 6).
- **Autres facteurs utilisés** : `filiere` , `bac_type` , `mention_bac` , `etablissement_origine` — risque de biais systémique si une filière ou un type de bac est structurellement lié a des facteurs socio-économiques (chapitre 2.6).

Données

- **Source** : `dataset_decrochage_etudiants_complet_V5.csv` (5240 lignes, 33 colonnes, année 2024-2025) enrichi de `dataset_catalogue_formations_V5.csv` (8 filières) — chapitre 3.1.
- **Nettoyage** : 40 doublons exacts supprimés (5200 lignes utiles), jusqu'à 12 % de valeurs manquantes sur certaines colonnes, imputées par médiane **fit sur le train uniquement** (chapitres 7.11, 7.13 — anti-fuite train/test).
- **Split** : 80/20 stratifié sur `abandon` , `random_state=42` — 4160 lignes train, **1040 lignes test jamais vues à l'entraînement** (chapitre 7.13).
- **Déséquilibre de la cible** : 28.4 % d'abandon (chapitre 6.1), compensé par `class_weight="balanced"` pour le classifieur, pas de SMOTE.

Métriques (sur le test, 1040 étudiants)

Classifieur	<code>abandon</code> (seuil 0.5)	Régresseur	<code>moyenne_finale</code>
ROC AUC	0.946	R^2	0.692
Précision	0.729	MAE	2.25 points/20
Rappel	0.899	RMSE	3.05 points/20
F1	0.805		

Détail complet (comparaison de modèles, validation croisée, calibration, SHAP) : chapitres 8, 9.1-9.4.
 Seuil de décision : **0.5 par défaut** (coïncide avec l'optimum F1) ; alternative **0.33** chiffrée en euros au chapitre 14.3 (économie nette supérieure mais ROI ratio inférieur — arbitrage non tranché avec la Direction).

Considérations éthiques

- RGPD : identifiants (`student_id` , `id_dossier`) exclus ; base légale du traitement non encore validée avec le DPO (chapitre 11.2.1).
- Dilemme du marquage / stigmatisation : un score n'est jamais montré tel quel, doit rester une aide à la décision humaine, jamais une décision automatique (chapitres 2.9, 11.2.2).
- Explicabilité : SHAP global (chapitre 9.2.1) et individuel (chapitre 12.1) disponibles pour justifier chaque signalement auprès d'un conseiller.

Limites et recommandations

- Aucune mesure de l'efficacité réelle d'un accompagnement déclenché sur la base d'un signalement — hypothèse optimiste implicite dans le chiffrage ROI (chapitre 14.3).
- Biais résiduel sur les variables sensibles non mesuré empiriquement (recommandation 6, chapitre 14.2).
- Modèle non testé sur une autre cohorte/année : la dérive doit être surveillée (Prometheus/Grafana, chapitre 10.6) et le modèle ré-entraîné selon le calendrier académique (chapitre 13.1).
- Pas de gate humain avant promotion d'une nouvelle version en production (recommandation 2, chapitre 14.2).